

IF3170 INTELIGENSI ARTIFISIAL

**PENCARIAN SOLUSI PENJADWALAN KELAS MINGGUAN
DENGAN LOCAL SEARCH**

Laporan Tugas Besar 1

Disusun untuk memenuhi tugas mata kuliah Inteligensi Artifisial pada Semester 5
(lima)

Tahun Akademik 2025/2026



Oleh:

Brian Ricardo Tamin	13523126
Muhamad Nazih Najmudin	13523144
Lukas Raja Agripa	13523158

AI_anak_indonesia

**PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
BANDUNG
2025**

DAFTAR ISI

DAFTAR ISI

BAB I

- 1.1 Latar Belakang Masalah
- 1.2 Spesifikasi Program
- 1.3 Realisasi

BAB II

- 2.1 Pembahasan
- 2.2 Pemilihan Objective Function
 - 2.2.1 Kasus Mahasiswa Bentrok
 - 2.2.2 Kasus Kapasitas Kurang
 - 2.2.3 Kasus Tidak Ada Dosen
 - 2.2.4 Kasus Dosen Bentrok
- 2.3 Fungsi dan Kelas
 - 2.3.1 Class Mahasiswa
 - 2.3.2 Class Ruangan
 - 2.3.3 Class Waktu
 - 2.3.4 Class MataKuliah
 - 2.3.5 Class Dosen
 - 2.3.6 Class State
 - 2.3.6 Class LocalSearch (Abstract Class)
 - 2.3.6 Class GeneticAlgorithm (extend LocalSearch)
 - 2.3.7 Class HillClimbing (extends LocalSearch)
 - 2.3.8 Class Simulated Annealing
 - 2.3.9 Class AlgorithmConfigServices
 - 2.3.10 Class OutputConfigServices
 - 2.3.11 Class AlgorithmExecutionServices
 - 2.3.12 Class OutputService
 - 2.3.13 Class UserInterfacesServices
 - 2.3.14 Class Table
 - 2.3.15 Class Info
 - 2.3.16 Class JsonParser
 - 2.3.17 Class Repository
- FUNGSI TAMBAHAN
 - 2.3.18 Settings
 - 2.3.19 Main dan Run
- 2.4 Implementasi Algoritma Local Search
 - 2.4.1 Algoritma Hill Climbing
 - 2.4.2 Algoritma Simulated Annealing
 - 2.4.3 Algoritma Genetik

BAB III

3.1 Eksperimen

3.1.1 Hill Climbing

3.1.2 Simulated Annealing

3.1.3 Genetic Algorithm

3.2 Analisis

BAB IV

4.1 Kesimpulan

4.2 Saran

LAMPIRAN

REFERENSI

BAB I

DESKRIPSI PERSOALAN

1.1 Latar Belakang Masalah

Penjadwalan mata kuliah merupakan aspek krusial dalam pengelolaan institusi pendidikan tinggi, karena memastikan efisiensi penggunaan sumber daya dan kelancaran proses belajar mengajar. Namun, proses manual sering kali menghadapi kesulitan yang signifikan, seperti konflik jadwal mahasiswa yang menyebabkan bentrok antara mata kuliah wajib dan pilihan, serta ketersediaan dosen yang terbatas akibat beban mengajar atau kegiatan lain, sehingga berpotensi menimbulkan ketidakefisienan dan ketidakpuasan di kalangan stakeholders.

Untuk mengatasi tantangan tersebut, pengembangan sistem penjadwalan berbasis local search menjadi solusi inovatif yang dapat mengoptimalkan alokasi waktu dan ruang. Sistem ini memperhitungkan faktor kapasitas ruangan agar sesuai dengan jumlah mahasiswa, menghindari overcrowding, dan secara otomatis menyelesaikan konflik, sehingga meningkatkan kualitas pendidikan secara keseluruhan.

1.2 Spesifikasi Program

Spesifikasi program adalah untuk membuat jadwal kelas selama satu minggu untuk sekumpulan mata kuliah yang harus dipasangkan dengan waktu dan tempat (ruang/kelas).

1.2.1 Daftar Variabel

Entitas	Atribut	Contoh	Keterangan
Kelas Mata kuliah	Kode	IF3071_K01	Kode mata kuliah dan kode kelas digabung.
	Jumlah mahasiswa terdaftar	60	Jumlah mahasiswa 60 hanya untuk satu kelas saja (dalam kasus ini untuk K01 sesuai kode di atas).
	Jumlah SKS	3	
Waktu	Waktu mulai	[Senin, 11]	Jam dalam integer. 1 SKS = 1 jam. Struktur data gabungan harus list .
	Waktu akhir	[Selasa, 13]	

Ruangan	Kode ruangan	7609	Untuk ruangan yang tidak memiliki kode, gunakan nama ruangan sebagai kode (misalnya “multimedia”).
	Kuota ruangan	60	Ruangan tersebut dapat menampung 60 mahasiswa.
Mahasiswa	NIM	13523999	
	Daftar mata kuliah	[IF3071_K01, IF9999_K99]	Bentuk data bebas, bisa menggunakan dictionary, dua list, atau implementasi lain.
	Prioritas mata kuliah	[1,2]	Elemen harus unik (satu orang hanya memiliki 1 matkul untuk prioritas tertentu).

Ketentuan Representasi

- Struktur data yang digunakan untuk representasi bebas. Misal, menggunakan key-value seperti JSON.
 - Representasi solusi (**state**) adalah pemetaan setiap Pertemuan Mata Kuliah ke Waktu dan Ruangan.
 - Inisialisasi (**initial state**) random.
 - Langkah neighbor (**move**) yang diperbolehkan tiap iterasi:
 1. menukar posisi dua pertemuan mata kuliah
 2. memindahkan pertemuan mata kuliah ke waktu dan ruangan yang kosong
- *Pertemuan Mata Kuliah merupakan kombinasi dari Kelas Mata Kuliah, Waktu, dan Ruangan sesuai dengan jumlah SKS yang dimiliki. Misalnya IF3071_K01 dengan 3 sks memiliki 2 Pertemuan Mata Kuliah, kelas di jam 7-9 hari Senin di ruang 7609 dan kelas di jam 9-10 hari Selasa di ruang multimedia.*

Contoh Input

```

{
  "kelas_mata_kuliah": [
    {
      "kode": "IF3071_K01",
      "jumlah_mahasiswa": 60,
      "sks": 3
    },
    {
      "kode": "IF3130_K01",
      "jumlah_mahasiswa": 45,
      "sks": 2
    },
    {
      "kode": "IF3110_K02",
      "jumlah_mahasiswa": 70,
      "sks": 3
    },
    {
      "kode": "IF3140_K01",
      "jumlah_mahasiswa": 55,
      "sks": 2
    }
  ],
  "ruangan": [
    {
      "kode": "7609",
      "kuota": 60
    },
    {
      "kode": "7606",
      "kuota": 80
    },
    {
      "kode": "multimedia",
      "kuota": 40
    }
  ],
  "mahasiswa": [
    {
      "nim": "13523601",
      "daftar_mk": ["IF3071_K01", "IF3130_K01"],
      "prioritas": [1, 2]
    },
    {
      "nim": "135236641",
      "daftar_mk": ["IF3110_K02", "IF3130_K01"],
      "prioritas": [1, 2]
    }
  ]
}

```

```

    },
    {
      "nim": "13523669",
      "daftar_mk": ["IF3140_K01", "IF3071_K01"],
      "prioritas": [1, 2]
    },
    {
      "nim": "13523600",
      "daftar_mk": ["IF3110_K02"],
      "prioritas": [1]
    }
  ],

```

Contoh Output

Kode ruang: 7609

Jam	Senin	Selasa	Rabu	Kamis	Jumat
7				IF3140	
8	IF3130		IF3110		IF3130
9	IF3130	IF3140	IF3110		
10		IF3140	IF3110		
11					
12					
13					
14					
15					
16					
17					

**kolom jam di output merupakan jam mulai*

**contoh di atas merupakan output untuk satu ruangan.*

1.2.2. Objective Function

Objective function yang digunakan **dibebaskan** (minimal 1), namun tetap berpedoman pada pada acuan berikut:

1. **Jumlah Waktu yang bertabrakan untuk tiap Mahasiswa**

Misalnya, mahasiswa A memiliki 2 mata kuliah di jam 9-11 hari Selasa akan berkontribusi 2 poin ke state tersebut.

2. **Jumlah Pertemuan Beberapa Mata Kuliah yang bertabrakan di Ruang dan Waktu tertentu dikali dengan bobot prioritas Mata Kuliah tiap peserta kelas**
Misalnya, IF2230_K01 dan IF1221_K01 sama-sama dilaksanakan di ruangan 7606 pada jam 13 - 14 . Bobot prioritas matkul adalah sebagai berikut.

Prioritas	Bobot
1	1.75
2	1.5
3	1.25
dll	1

Peserta kelas IF2330_K01 adalah sebagai berikut.

```
{
  "nim": "13523601",
  "daftar_mk": ["IF2330_K01"],
  "prioritas": [1]
},
{
  "nim": "135236641",
  "daftar_mk": ["IF2330_K01"],
  "prioritas": [2]
}
```

Sedangkan peserta kelas IF1221_K01 adalah sebagai berikut.

```
{
  "nim": "13523701",
  "daftar_mk": ["IF1221_K01"],
  "prioritas": [1]
},
{
  "nim": "135237641",
  "daftar_mk": ["IF1221_K01"],
  "prioritas": [3]
}
```

Maka state valuenya akan bertambah/berkurang (tergantung implementasi) sebanyak
 $1 \text{ jam} * (1.75 \text{ bobot prioritas } 1 * 2 \text{ jumlah mahasiswa prioritas } 1 + 1.5 \text{ bobot prioritas } 2 * 1 \text{ jumlah mahasiswa prioritas } 2 + 1.25 \text{ bobot prioritas } 3 * 1 \text{ jumlah mahasiswa prioritas } 3 + 1 \text{ bobot prioritas lainnya} * 0 \text{ jumlah mahasiswa prioritas lainnya}) = 6.25$

3. **Selisih Jumlah Mahasiswa di Pertemuan Mata Kuliah tertentu dengan Kuota Ruang tersedia (dalam kasus Jumlah Mahasiswa > Kuota Ruang)**

Misalnya terdapat 65 mahasiswa di ruangan 7609 (Kuota Ruangan: 60) di suatu Pertemuan Mata Kuliah **2 SKS** akan berkontribusi 10 poin $((65 - 60) * 2)$ ke state tersebut.

1.2.3. Cakupan

Anda akan diminta untuk mengimplementasikan rencana yang telah Anda buat pada Tugas Besar 1. Berikut merupakan hal-hal yang perlu dilakukan oleh setiap kelompok:

- Implementasikan 3 algoritma local search dengan rincian sebagai berikut:
 - Salah satu algoritma hill-climbing
 - Simulated Annealing
 - Genetic Algorithm
- Lakukan eksperimen dengan skema sebagai berikut:
 - Jalankan setiap algoritma sebanyak **3 kali**, kemudian catat beberapa hal berikut:
 - Berlaku untuk semua algoritma
 - State awal dan akhir
 - Nilai objective function akhir yang dicapai
 - Plot nilai objective function terhadap banyak iterasi yang telah dilewati
 - Durasi proses pencarian
 - Berlaku hanya untuk Steepest Ascent Hill-Climbing dan Stochastic Hill-Climbing
 - Banyak iterasi hingga proses pencarian berhenti
 - Berlaku hanya untuk Hill-Climbing with Sideways Move
 - Banyak iterasi hingga proses pencarian berhenti
 - Note: Tambahkan parameter maximum sideways move, dimana ketika banyak sideways move yang dilakukan sudah mencapai maksimum, pencarian dihentikan
 - Berlaku hanya untuk Random Restart Hill-Climbing
 - Banyak restart
 - Banyak iterasi per restart
 - Note: Tambahkan parameter maximum restart, dimana ketika banyak restart sudah mencapai maksimum, pencarian dihentikan.
 - Berlaku hanya untuk Simulated Annealing
 - Plot $e^{\frac{\Delta E}{T}}$ terhadap banyak iterasi yang telah dilewati
 - Frekuensi 'stuck' di local optima
 - Khusus untuk Genetic Algorithm, lakukan beberapa hal berikut:
 - Terdapat 2 parameter yang dapat diubah, yaitu **jumlah populasi** dan **banyak iterasi**.

- Jadikan jumlah populasi sebagai kontrol, kemudian pilih **3 variasi** banyak iterasi yang berbeda. Jalankan program sebanyak masing-masing **3 kali** untuk setiap konfigurasi parameter.
- Jadikan banyak iterasi sebagai kontrol, kemudian pilih **3 variasi** jumlah populasi yang berbeda. Jalankan program sebanyak masing-masing **3 kali** untuk setiap konfigurasi parameter.
- Untuk setiap eksperimen, catat beberapa hal berikut:
 - State awal dan akhir
 - Nilai objective function akhir yang dicapai
 - Plot nilai objective function terhadap banyak iterasi yang telah dilewati (Cukup plot nilai objective function maksimum dan rata-rata dari populasi terhadap banyak iterasi yang telah dilewati. Jika sudah terlanjur membuat plot untuk tiap individu pada populasi tidak menjadi masalah, silahkan diberikan keterangan saja maksud plotnya apa)
 - Jumlah populasi
 - Banyak iterasi
 - Durasi proses pencarian
- Lakukan analisis terhadap hasil eksperimen, berikut merupakan beberapa pertanyaan yang dapat menjadi acuan untuk analisis yang Anda lakukan (Anda boleh menambahkan beberapa pertanyaan tambahan jika dirasa perlu untuk dijelaskan):
 - Seberapa dekat tiap-tiap algoritma bisa mendekati global optima dan mengapa hasilnya demikian?
 - Bagaimana perbandingan hasil pencarian tiap-tiap algoritma dengan algoritma local search yang lain?
 - Bagaimana perbandingan durasi proses pencarian tiap algoritma relatif terhadap algoritma lainnya?
 - Seberapa konsisten hasil akhir yang didapatkan dari tiap-tiap eksperimen yang dilakukan?
 - Bagaimana pengaruh banyak iterasi dan jumlah populasi terhadap hasil akhir pencarian pada Genetic Algorithm?
 - dst...
- Program yang dibuat harus bisa **memvisualisasikan** state awal, state akhir, dan juga hasil eksperimennya (sesuaikan informasi hasil eksperimen yang ditampilkan dengan ketentuan yang telah dijelaskan di poin sebelum ini).
- Cara visualisasi dibebaskan kepada masing-masing kelompok.
- Gunakan bahasa **Python**.
- Diperbolehkan untuk menggunakan heuristik yang Anda buat sendiri atau dari referensi lain untuk optimasi pencarian solusi, asalkan masih dalam lingkup local search. Jangan lupa jelaskan heuristik yang Anda pakai di laporan.

1.2.4. Spesifikasi Bonus

Hanya kerjakan spesifikasi ini jika Anda sudah menyelesaikan seluruh spesifikasi wajib. Berikut merupakan beberapa spesifikasi bonus yang dapat Anda kerjakan:

- Buatlah implementasi untuk **seluruh algoritma hill-climbing** yang ada.
- Tambahkan **jadwal dosen** sebagai komponen objective function (jadwal dosen dikreasikan sendiri).

1.3 Realisasi

Berdasarkan spesifikasi yang diberikan, pengamat mengerjakan keseluruhan spesifikasi yang diberikan, termasuk bonus, dengan bahasa pemrograman *Python* dengan paradigma berbasis objek melalui tampilan output dalam terminal dan juga tampilan plot output dengan *library Matplotlib*.

BAB II

PEMBAHASAN

2.1 Pembahasan

Pada tugas besar ini, algoritma yang digunakan untuk menyelesaikan permasalahan penjadwalan mata kuliah, digunakan tiga jenis algoritma, yaitu algoritma Hill Climbing yang terdiri dari empat variasi, yakni steepest ascent, sideways, stochastic, dan random restart. selain itu ada pula algoritma Simulated Annealing dan algoritma Genetic Algorithm. Seluruh algoritma ini menggunakan representasi state berupa pemetaan slot(ruangan, waktu(hari, jam)) terhadap mata kuliah yang di-assign ke slot tersebut. Suatu state dikatakan lebih baik dari state lain apabila state valuenya lebih besar dibanding state lain tersebut. Adapun hasil sempurna yang diharapkan adalah state dengan state value sebesar nol. State value diperoleh dari negasi dari total penjumlahan hasil perhitungan yang berdasarkan fungsi-fungsi objektif yang tertera sebagai berikut.

2.2 Pemilihan Objective Function

2.2.1 Kasus Mahasiswa Bentrok

Fungsi objektif ini menghitung beban yang diperoleh sebuah state apabila terdapat dua atau lebih mata kuliah yang dijadwalkan pada waktu yang sama dan ruangan yang berbeda sedemikian sehingga terdapat mahasiswa yang mendaftar dua atau lebih mata kuliah tersebut sehingga berbentrok. Beban dihitung berdasarkan mahasiswa yang jadwalnya berbentrok dengan pembobotan sesuai prioritasnya untuk setiap mata kuliah yang berbentrok tersebut.

```

def kasus_mahasiswa_bentrok(state: State) -> float:
    """Menghitung beban untuk matakuliah yang bentrok berdasarkan prioritas mahasiswa"""
    list_matkul_bentrok = [
        list_matkul for _, list_matkul in state.timesToMK.items() if len(list_matkul) > 1
    ]
    beban_state = 0.0
    for matkul_bentrok in list_matkul_bentrok:
        mahasiswa_mungkin_bentrok = [
            set(state.repo.mahasiswa_tiap_matkul[matkul.kode]) for matkul in matkul_bentrok
        ]
        mahasiswa_bentrok = set.intersection(*mahasiswa_mungkin_bentrok)
        if not mahasiswa_bentrok:
            continue

        for mahasiswa in mahasiswa_bentrok:
            for matkul in matkul_bentrok:
                try:
                    idx = mahasiswa.mata_kuliah.index(matkul.kode)
                    prioritas = mahasiswa.prioritas[idx]
                except ValueError:
                    prioritas = 4 # Default priority if not found

                if prioritas == 1:
                    beban_state += 1.75
                elif prioritas == 2:
                    beban_state += 1.5
                elif prioritas == 3:
                    beban_state += 1.25
                else:
                    beban_state += 1
    return beban_state

```

2.2.2 Kasus Kapasitas Kurang

Fungsi objektif ini menghitung beban yang diperoleh sebuah state apabila terdapat mata kuliah dengan total mahasiswa yang melebihi kapasitas ruangnya. Beban dihitung berdasarkan jumlah mahasiswa yang tidak mendapat tempat duduk di mata kuliah tersebut dikalikan dengan jumlah sks mata kuliahnya.

```

def kasus_kapasitas_kurang(state: State) -> int:
    """Menghitung beban untuk setiap slot yang ditempati oleh
    matakuliah dengan pendaftar melebihi kapasitas"""

    beban_state = 0
    for slot, matkul in state.assignments.items():
        jumlah_mahasiswa = len(state.repo.mahasiswa_tiap_matkul[matkul.kode])
        kapasitas_ruangan = state.repo.ruangan[slot[0]].kuota_ruangan
        sks_matakuliah = matkul.jumlah_sks
        beban_state += max(0, (jumlah_mahasiswa - kapasitas_ruangan)*sks_matakuliah)
        # beban_state += max(0, (jumlah_mahasiswa - kapasitas_ruangan))

    return beban_state

```

2.2.3 Kasus Tidak Ada Dosen

Fungsi objektif ini menghitung beban yang diperoleh sebuah state apabila terdapat mata kuliah yang tidak dapat dihadiri oleh satu dosen pun karena tidak ada dosen yang bersedia mengajar pada waktu tersebut. Beban dihitung berdasarkan total mahasiswa yang terdampak akibat ketidaktersedianya dosen pada mata kuliah tersebut.

```
def kasus_dosen_gabisa(state:State) -> int:
    """Menghitung beban matakuliah yang tidak bisa diajar oleh dosen pada waktu tersebut"""
    beban_state = 0
    for slot, matkul in state.assignments.items():
        dosen_pengampu = state.repo.dosen_tiap_matkul[matkul.kode]
        add_beban_state = 0
        for dosen in dosen_pengampu:
            # Minimal ada 1 dosen yang bisa mengajar
            if slot[1] in dosen.waktu_preferensi:
                add_beban_state += 0
                break
            else:
                add_beban_state += len(state.repo.mahasiswa_tiap_matkul[matkul.kode])
        beban_state += add_beban_state

    return beban_state
```

2.2.4 Kasus Dosen Bentrok

Fungsi objektif ini menghitung beban yang diperoleh sebuah state apabila terdapat mata kuliah yang tidak mendapat dosen karena terdapat dua atau lebih mata kuliah yang diajar oleh dosen yang sama yang dijadwalkan pada waktu yang sama. Kasus diperbolehkan apabila terdapat minimal satu dosen yang dapat mengajar kelas tersebut. Beban dihitung berdasarkan jumlah mahasiswa tiap matkul yang terdampak.

```
def kasus_dosen_bentrok(state: State) -> int:
    """Menghitung beban dosen yang matkulnya berbentrok"""
    list_matkul_bentrok = [
        list_matkul for _, list_matkul in state.timesToMK.items() if len(list_matkul) > 1
    ]
    beban_state = 0
    for matkul_bentrok in list_matkul_bentrok:
        dosen_mungkin_bentrok = [
            set(state.repo.dosen_tiap_matkul[matkul.kode]) for matkul in matkul_bentrok
        ]
        # Compare all pairs of lecturer sets for conflicts
        for i in range(len(dosen_mungkin_bentrok)):
            for j in range(i + 1, len(dosen_mungkin_bentrok)):
                if not dosen_mungkin_bentrok[i].isdisjoint(dosen_mungkin_bentrok[j]):
                    # Add the number of students enrolled in both conflicting courses
                    beban_state += len(state.repo.mahasiswa_tiap_matkul[matkul_bentrok[i].kode])
                    beban_state += len(state.repo.mahasiswa_tiap_matkul[matkul_bentrok[j].kode])
    return beban_state
```

2.3 Fungsi dan Kelas

2.3.1 Class Mahasiswa

2.3.1.1 Atribut

Nama Atribut	Penjelasan Singkat
<code>nim (string)</code>	NIM Mahasiswa
<code>mata_kuliah List[string]</code>	Mata Kuliah Mahasiswa
<code>prioritas List[integer]</code>	Prioritas Mahasiswa

3.1.1.2 Metode

Nama Metode	Penjelasan Singkat
<pre>def __hash__(self): return hash(self.nim)</pre>	fungsi hashable pada nim tiap objek mahasiswa, karena nim adalah unik
<pre>def __eq__(self, other): if not isinstance(other, Mahasiswa): return False return self.kode == other.nim</pre>	fungsi checking nim antar objek apakah sama atau tidak.

2.3.2 Class Ruangan

2.3.2.1 Atribut

Nama Atribut	Penjelasan Singkat
<code>kode_ruangan (string)</code>	Kode Ruangan
<code>kuota_ruangan (integer)</code>	Kuota Ruangan

3.1.2.2 Metode

Nama Metode	Penjelasan Singkat
Tidak ada method	

2.3.3 Class Waktu

2.3.3.1 Atribut

Nama Atribut	Penjelasan Singkat
hari (string)	Hari
jam (integer)	Jam

3.1.3.2 Metode

Nama Metode	Penjelasan Singkat
Tidak ada method	

2.3.4 Class MataKuliah

2.3.4.1 Atribut

Nama Atribut	Penjelasan Singkat
kode (string)	Kode Mata Kuliah
jumlah_mahasiswa (integer)	Jumlah Mahasiswa yang terdaftar
jumlah_sks (integer)	Jumlah SKS

3.1.4.2 Metode

Nama Metode	Penjelasan Singkat
<pre>def __hash__(self): return hash(self.nim)</pre>	fungsi hashable pada kode tiap objek mahasiswa, karena nim adalah unik
<pre>def __eq__(self, other): if not isinstance(other, MataKuliah): return False return self.kode == other.kode</pre>	fungsi checking kode antar objek apakah sama atau tidak.

2.3.5 Class Dosen

2.3.5.1 Atribut

Nama Atribut	Penjelasan Singkat
<code>nama (string)</code>	Nama Dosen
<code>mata_kuliah_diampu List[string]</code>	Mata Kuliah yang diampu oleh Dosen
<code>waktu_preferensi List[Waktu]</code>	Waktu yang dapat digunakan dosen untuk mengajar

3.1.5.2 Metode

Nama Metode	Penjelasan Singkat
<pre>def __hash__(self): return hash(self.nama)</pre>	fungsi hashable pada nim tiap objek mahasiswa, karena nama dosen adalah unik
<pre>def __eq__(self, other): if not isinstance(other, Mahasiswa): return False return self.kode == other.nama</pre>	fungsi checking nama dosen antar objek apakah sama atau tidak.

2.3.6 Class State

2.3.1.1 Atribut

Nama Atribut	Penjelasan Singkat
<code>repo (Repository)</code>	Referensi ke repository sebagai acuan data.
<code>availableSlots (Set[Tuple[str, Waktu]])</code>	Domain Slot(ruangan, waktu) yang valid.
<code>assignments: Dict[Tuple[str, Waktu], MataKuliah]</code>	Pemetaan Slot(ruangan, waktu) dengan mata kuliah yang di-assign ke slot tersebut.
<code>timesToMK (Dict[Waktu, List[MataKuliah]])</code>	Pemetaan waktu dengan seluruh mata kuliah yang di-assign di waktu

	tersebut.
<code>MKtoSlots (Dict[str, List[Tuple[str, Waktu]])])</code>	Pemetaan mata kuliah dengan slot-slot yang di-assign olehnya.
<code>stateValue (float)</code>	State value $(-\infty, 0]$ dari sebuah state. Semakin besar semakin baik.

3.1.1.2 Metode

Nama Metode	Penjelasan Singkat
<code>def initializeDomain(self, repo: Repository, _withDosen:bool=True):</code>	fungsi inialisasi domain berdasarkan repositori yang telah di-load dari json.
<code>def copy(self) -> "State":</code>	fungsi menyalin data secara semi-shallow copy, yaitu mengcopy hingga tingkat koleksi, tetapi tidak mengcopy objek data.
<code>def assignMK(self, kode_mk: str, kode_ruangan: str, waktu: Waktu):</code>	fungsi assignment atau menambahkan matakuliah ke slot(ruangan, waktu) pada jadwal yang kosong.
<code>def removeMK(self, kode_ruangan: str, waktu: Waktu) -> Optional[MataKuliah]:</code>	fungsi menghapus matakuliah dari assignment.
<code>def swapMK(self, slot1: Tuple[str, Waktu], slot2: Tuple[str, Waktu]):</code>	fungsi menukar dua plottingan mata kuliah, dapat ditukar dengan slot yang sudah terisi maupun slot yang masih kosong. menukar dengan slot kosong sama saja dengan memindahkan assignment.
<code>def countStateValue(self, *objectives: Callable) -> float</code>	fungsi menghitung state value terbaru berdasarkan objective function.
<code>def generateAllSuccessors(self, *objectives) -> list["State"]:</code>	fungsi membangkitkan seluruh suksesor yang mungkin.

<pre>def generateRandomSuccessor(self, *objectives) -> "State":</pre>	fungsi menghasilkan satu buah suksesor acak tanpa membangkitkan seluruh suksesor terlebih dahulu.
<pre>def initializeRandomSuccessor(self, *objectives: Callable): def autoSortedAppend(myList: List[Tuple[str, Waktu]], newItem: Tuple[str, Waktu], firstOrder: List[str], hariOrder: List[str] = ["Senin", "Selasa", "Rabu", "Kamis", "Jumat"]) -> None:</pre>	fungsi inisialisasi state pertama secara random.

2.3.6 Class LocalSearch (Abstract Class)

2.3.6.1 Atribut

Nama Atribut	Penjelasan Singkat
Tidak ada atribut	

3.1.6.2 Metode

Nama Metode	Penjelasan Singkat
<pre>def search(self, state: State, *objectives): pass</pre>	fungsi pencarian yang harus diimplementasi.

2.3.6 Class GeneticAlgorithm (extend LocalSearch)

2.3.6.1 Atribut

Nama Atribut	Penjelasan Singkat
state (State)	State awal yang akan diproses
jml_parent (integer)	Jumlah populasi per generasi
jml_iterasi (integer)	Jumlah iterasi

3.1.6.2 Metode

Nama Metode	Penjelasan Singkat
<code>def _initialize_availableTime_for_matkul(self):</code>	fungsi pemetaan matakuliah dengan waktu yang mungkin untuk di-assign berdasarkan ketersediaan dosen.
<code>def _initialize_conf_crossover(self, conf_crossover=None):</code>	fungsi inisialisasi konfigurasi strategi crossover.
<code>def search(self, state:State, *objectives:Callable, conf_crossover=None) -> Tuple[State, Dict[str, Any]]:</code>	fungsi pencarian solusi.
<code>def _initialize_parent(self, *objectives) -> List[State]:</code>	fungsi pembangkitan parent awal secara random.
<code>def _crossover(self, stateA:State, stateB:State, *objectives:Callable) -> Tuple[State, State]:</code>	fungsi crossover sesuai konfigurasi.
<code>def _mutation(self, state:State, *objectives:Callable) -> State:</code>	fungsi mutasi berupa pembangkitan random successor.

2.3.7 Class HillClimbing (extends LocalSearch)

2.3.7.1 Atribut

Nama Atribut	Penjelasan Singkat
<code>var (string)</code>	Variasi jenis HC
<code>max_sideways (integer)</code>	Banyak maksimum penerimaan bahu curva pada variasi sideways
<code>max_iteration (integer)</code>	Banyak maksimum iterasi pada variasi stochastic

<code>max_restart (integer)</code>	Banyak restart maksimum pada varias random restart
------------------------------------	--

3.1.7.2 Metode

Nama Metode	Penjelasan Singkat
<pre>def search(self, state: State, *objectives) -> Tuple[State, Dict[str, Any]]:</pre>	fungsi end point algoritma HC yang merujuk sesuai variant nya.
<pre>def _steepest(self, state: State, *objectives) -> Tuple[State, Dict[str, Any]]:</pre>	fungsi pencarian steepest ascent HC.
<pre>def _sideways(self, state: State, MAX_SIDEWAYS: int, *objectives) -> Tuple[State, Dict[str, Any]]:</pre>	fungsi pencarian sideways HC.
<pre>def _stochastic(self, state: State, iterations: int, *objectives) -> Tuple[State, Dict[str, Any]]:</pre>	fungsi pencarian stochastic HC.
<pre>def _random_restart(self, state: State, restarts:int, *objectives) -> Tuple[State, Dict[str, Any]]:</pre>	fungsi pencarian random restart HC.

2.3.8 Class Simulated Annealing

2.3.8.1 Atribut

Nama Atribut	Penjelasan Singkat
<code>initialT (int)</code>	Temperatur inisial
<code>coolingRate (int)</code>	Kecepatan penurunan
<code>stopT (int)</code>	Temperature berhenti

3.1.8.2 Metode

Nama Metode	Penjelasan Singkat
<code>def setInitialTemperature(self, initialT: float):</code>	fungsi inisiasi temperatur awal.
<code>def setCoolingRate(self, coolingRate: float):</code>	fungsi pengaturan cooling rate.
<code>def setStopTemperature(self, stopT: float):</code>	fungsi pengaturan temperatur yang menandakan terminasi.
<code>def _acceptance_probability(self, _deltaE: float, temperature: float) -> float:</code>	fungsi penghitungan probabilitas penerimaan state yang state valuenya lebih rendah dari state saat ini.
<code>def search(self, state: State, *objectives) -> Tuple[State, Dict[str, Any]]:</code>	fungsi pencarian.

2.3.9 Class AlgorithmConfigServices

2.3.9.1 Atribut

Nama Atribut	Penjelasan Singkat
Tidak ada atribut	

3.1.9.2 Metode

Nama Metode	Penjelasan Singkat
<code>def configureHillClimbing() -> Dict[str, Any]:</code>	fungsi antarmuka User untuk menerima input konfigurasi user pada Algoritma HC
<code>def configureSimulatedAnnealing() -> Dict[str, Any]:</code>	fungsi antarmuka User untuk menerima input konfigurasi user pada Algoritma SA
<code>def configureGeneticAlgorithm() -> Dict[str, Any]:</code>	fungsi antarmuka User untuk menerima input konfigurasi user pada Algoritma GA

2.3.10 Class OutputConfigServices

2.3.10.1 Atribut

Nama Atribut	Penjelasan Singkat
Tidak ada atribut	

3.1.10.2 Metode

Nama Metode	Penjelasan Singkat
<code>def askSaveOption() -> bool:</code>	fungsi save output
<code>def generateFilename(algorithmName: str) -> str:</code>	fungsi generate nama file output ketika save
<code>def showSaveSuccess(filename: str): Dict[str, Any] -> Tuple[State, Dict[str, Any]]:</code>	fungsi menampilkan pesan ke user bahwa berhasil di save (ternasuk savenya)

2.3.11 Class AlgorithmExecutionServices

2.3.11.1 Atribut

Nama Atribut	Penjelasan Singkat
Tidak ada atribut	

3.1.11.2 Metode

Nama Metode	Penjelasan Singkat
def runHillClimbing(initialState: State, config: Dict[str, Any]) -> Tuple[State, Dict[str, Any]]:	fungsi keseluruhan meluncurkan algoritma HC (driver)
def runSimulatedAnnealing(initialState: State, config: Dict[str, Any]) -> Tuple[State, Dict[str, Any]]	fungsi keseluruhan meluncurkan algoritma SA (driver)
def runGeneticAlgorithm(initialState: State, config: Dict[str, Any]) -> Tuple[State, Dict[str, Any]]:	fungsi keseluruhan meluncurkan algoritma GA (driver)

2.3.12 Class OutputService

2.3.12.1 Atribut

Nama Atribut	Penjelasan Singkat
Tidak ada atribut	

3.1.12.2 Metode

Nama Metode	Penjelasan Singkat
def generateAlgorithmInfo(algorithmName: str, parameters: Dict[str, Any], executionStats: Dict[str, Any]) -> Info:	fungsi manager perihal Output Info Algoritma setelah dieksekusi
def generateScheduleAnalysis(state: State) -> Info:	fungsi manager perihal Output jadwal setelah dieksekusi

<pre>def generateScheduleTables(s tate: State) -> Dict[str, Table]:</pre>	fungsi manager perihal Output Tabek setelah dieksekusi
<pre>def generateGraph(algorithmN ame: str, executionStats: Dict[str, Any]) -> None:</pre>	fungsi manager perihal Output Plot Info setelah dieksekusi
<pre>def displayResults(state: State, algorithmName: str, parameters: Dict[str, Any], executionStats: Dict[str, Any], initialState: State = None) -> None:</pre>	fungsi manager keseluruhan yang combine fungsi output diatas.
<pre>def showWelcome() -> None:</pre>	pesan antarmuka user
<pre>def showOptimizationStart() -> None:</pre>	antarmuka user untuk melihat proses yang dijalankan
<pre>def showResultHeader() -> None:</pre>	fungsi antarmuka header (mempercantik tampilan)
<pre>def saveResults(state: State, algorithmName: str, parameters: Dict[str, Any], executionStats: Dict[str, Any], filename: str, initialState: State = None) -> None:</pre>	fungsi driver save output
<pre>def _tableToString(table: Table) -> str:</pre>	mengubah object tabel ke string

2.3.13 Class UserInterfacesServices

2.3.13.1 Atribut

Nama Atribut	Penjelasan Singkat
Tidak ada atribut	

3.1.13.2 Metode

Nama Metode	Penjelasan Singkat
<code>def getValidFilePath() -> str:</code>	fungsi keseluruhan antarmuka user input file json
<code>def selectAlgorithm() -> str:</code>	fungsi keseluruhan user memilih algoiritma

2.3.14 Class Table

2.3.14.1 Atribut

Nama Atribut	Penjelasan Singkat
Tidak ada atribut	

3.1.14.2 Metode

Nama Metode	Penjelasan Singkat
<code>def setHeaders(self, headers: List[str]) -> None: # bikin header</code>	fungsi setter header
<code>def addRow(self, rowData: List[Any]) -> None: #tambah baris</code>	fungsi tambah row
<code>def display(self) -> None: # dari fungsi render dibawah diprintkan</code>	fungsi print ke terminal tabel yang dibuat
<code>def render(self) -> str:</code>	fungsi pembuat tabel

2.3.15 Class Info

2.3.15.1 Atribut

Nama Atribut	Penjelasan Singkat
Tidak ada atribut	

3.1.15.2 Metode

Nama Metode	Penjelasan Singkat
<pre>def addInfo(self, key: str, value: Any) -> None:</pre>	fungsi menambahkan info
<pre>def addSection(self, title: str, content: str) -> None:</pre>	fungsi menambahkan section.
<pre>def display(self) -> None:</pre>	fungsi menampilkan info
<pre>def render(self) -> str:</pre>	fungsi membuat format info

2.3.16 Class JsonParser

2.3.16.1 Atribut

Nama Atribut	Penjelasan Singkat
Tidak ada atribut	

3.1.16.2 Metode

Nama Metode	Penjelasan Singkat
<pre>def parse_section(self, key: str) -> list[dict]:</pre>	fungsi parsing section json.

2.3.17 Class Repository

2.3.17.1 Atribut

Nama Atribut	Penjelasan Singkat
<code>mata_kuliah:</code> Dict[str,	fungsi masing masing objek, mata

MataKuliah] ruangan: Dict[str, Ruangan] mahasiswa: Dict[str, Mahasiswa] dosen: Dict[str, Dosen] mahasiswa_tiap_matkul: Dict[str, List[Mahasiswa]] #str ke list mahasiswa nya tiap matkul dosen_tiap_matkul: Dict[str, List[Dosen]] #str ke list dosen nya tiap matkul	kuliah, ruangan, mahasiswa yang didapatkan dari JSON. Dan juga relasi tambahan seperti dosen dan mahasiswa pada setiap mata kuliah
--	---

3.1.17.2 Metode

Nama Metode	Penjelasan Singkat
def _to_waktu(raw_slot: list) -> Waktu	Dari list waktu menjadi objek Waktu
def create_repo(path: str) -> Repository	function utama dalam Repository untuk membuat sekumpulan objek dari JSON.

FUNGSI TAMBAHAN

2.3.18 Settings

2.3.14.1 Fungsi

Nama Fungsi	Penjelasan Singkat
def validPath(path: str) -> bool:	fungsi validasi path JSON dengan os library

2.3.19 Main dan Run

2.3.15.1 Fungsi

Nama Fungsi	Penjelasan Singkat
<code>def main() :</code>	menampung keseluruhan fungsi program
<pre> if __name__ == "__main__": # jika file dijalankan langsung main() # panggil fungsi main </pre>	driver

2.4 Implementasi Algoritma Local Search

2.4.1 Algoritma Hill Climbing

Algoritma	Garis Besar Cara Kerja
 <pre> 1 def steepest(self, state: State, *objectives) -> Tuple[State, Dict[str, Any]]: 2 """steepest ascent hill-climbing""" 3 current = state.copy() 4 current.initializeRandomSuccessor(*objectives) 5 initial_stateValue = current.stateValue 6 7 # Plot Hasil 8 stateValues = [current.stateValue] 9 iters = [0] 10 11 while True: 12 # generate successors 13 successors = current.generateAllSuccessors(*objectives) 14 if not successors: 15 break 16 17 # compare value 18 best_successor = max(successors, key=lambda s: s.stateValue) 19 stateValues.append(best_successor.stateValue) 20 iters.append(iters[-1]+1) 21 22 if best_successor.stateValue <= current.stateValue: 23 break 24 current = best_successor 25 26 # Plot Hasil 27 plotting_hasil = { 28 'MC Steepest Ascent': { 29 'x_label': 'Iterasi', 30 'y_label': 'State Value', 31 'x_data': iters, 32 'y_data': stateValues 33 } 34 } 35 36 stats = { 37 'iterations': iters[-1], 38 'initial_stateValue': initial_stateValue, 39 'final_stateValue': current.stateValue, 40 'plot_graph': plotting_hasil 41 } 42 43 return current, stats </pre>	1. initialize random state dari suatu given state 2. catat nilai awal initial state value 3. generate semua successor dari input parameter *objective untuk melakukan move dari current state 4. mencari value tertinggi dari best_successor menggunakan max() 5. jika successor yang di generate tidak ada yang melebihi current state value, maka return current state, dianggap sudah maxima (local / global)

```

1 def _sideways(self, state: State, MAX_SIDeways: int, *objectives) -> Tuple[State, Dict[str, Any]]:
2     """sideways variant hill-climbing"""
3     def _keys(s: State):
4         return frozenset(s.assignments.items())
5
6     current = state.copy()
7     current.initializeRandomSuccessor(*objectives)
8     initial_stateValue = current.stateValue
9
10    visited = (_keys(current))
11    sideways = 0
12
13    # Plot Hasil
14    stateValues = [current.stateValue]
15    iters = [0]
16
17    while True:
18        # generate successors
19        successors = current.generateAllSuccessors(*objectives)
20        if not successors:
21            break
22        current_value = current.stateValue
23        best_value = max(s.stateValue for s in successors)
24
25        # better value
26        if best_value > current_value:
27            better_states = [s for s in successors if s.stateValue == best_value]
28            current = random.choice(better_states)
29            visited.add(_keys(current))
30            sideways = 0
31
32            stateValues.append(current.stateValue)
33            iters.append(iters[-1]+1)
34            continue
35
36        # same value
37        if best_value == current_value and sideways < MAX_SIDeways:
38            equal_unvisited = [
39                s for s in successors
40                if s.stateValue == current_value and _keys(s) not in visited
41            ]
42            if equal_unvisited:
43                current = random.choice(equal_unvisited)
44                visited.add(_keys(current))
45                sideways += 1
46
47            stateValues.append(current.stateValue)
48            iters.append(iters[-1]+1)
49            continue
50        break
51
52    # Plot Hasil
53    plotting_hasil = {
54        "HC Sideways": {
55            "x_label": "Iterasi",
56            "y_label": "State Value",
57            "x_data": iters,
58            "y_data": stateValues
59        }
60    }
61
62    stats = {
63        "Iterations": iters[-1],
64        "Initial stateValue": initial_stateValue,
65        "Final stateValue": current.stateValue,
66        "Plot graph": plotting_hasil
67    }
68
69    return current, stats

```

1. membuat frozenset terlebih dahulu yang acts as hash set, supaya state yang sama tidak di kunjungi dan tidak bisa dimodifikasi
2. initialize random state menggunakan params *objective agar dapat di move untuk mencari state yang random
3. simpan current value dan best value dari successors
4. jika terdapat state yang valuenya lebih tinggi dari current_value dari state of successors, maka mengambil random state dari value tertinggi , ini tujuannya kalau misal ada multiple state dengan value yang sama, maka yang diambil adalah random.
5. sambil iterasi , sambil memvalidasi untuk memastikan bahwa pada hashset tidak ada state yang diambil kembali, dan mereset sideways jika memang ada state baru yang diambil dan lebih tinggi
6. sambil memvalidate jika jumlah sideways, dimana successors tidak ada yang valuenya lebih tinggi dari current, setelah nmax iterasi, maka return current, dan dianggap sudah maximum (local ataupun global)

```

1 def _stochastic(self, state: State, iterations: int, *objectives) -> Tuple[State, Dict[str, Any]]:
2     """stochastic hill-climbing"""
3     current = state.copy()
4     current.initializeRandomSuccessor(*objectives)
5     initial_stateValue = current.stateValue
6
7     # plot hasil
8     stateValues = [current.stateValue]
9     iters = [0]
10
11     steps = 0
12     while steps < iterations:
13         # if state value global max -> return
14         if current.stateValue == 0:
15             break
16
17         # generate successors
18         successor = current.generateRandomSuccessor(*objectives)
19
20         # if bigger accept
21         better = successor.stateValue > current.stateValue
22         if better:
23             current = successor
24
25         stateValues.append(current.stateValue)
26         iters.append(iters[-1]+1)
27
28         steps += 1
29
30     # Plot Hasil
31     plotting_hasil = {
32         'x_label': 'Iterasi',
33         'y_label': 'State Value',
34         'x_data': iters,
35         'y_data': stateValues
36     }
37
38     stats = {
39         'iterations': iters[-1],
40         'initial_stateValue': initial_stateValue,
41         'final_stateValue': current.stateValue,
42         'plot_graph': plotting_hasil
43     }
44
45     return current, stats

```

1. Start random neighbor. Mulai dari state.copy() lalu initialize Random Successor(*objectives) agar langsung di tetangga acak yang valid.
2. Satu sampel per langkah. Tiap iterasi ambil 1 generateRandomSuccessor(*objectives)
3. jika value lebih tinggi dari current , langsung accept satatenya
4. Early stop target. Jika current.stateValue = 0 (anggap goal tercapai).
5. Berhenti saat steps = iterations bila tak ada perbaikan yang diterima.

```

1 def _random_restart(self, state: State, restarts: int, *objectives) -> Tuple[State, Dict[str, Any]]:
2     """random_restart hill-climbing"""
3     best = None
4     stateValues = []
5     iters = []
6     restart = []
7     res_iter = []
8     res_val = []
9     initial_stateValue = 0
10
11     found = False
12     for i in range (restarts):
13
14         if found:
15             break
16
17         current = state.copy()
18         current.initializeRandomSuccessor(*objectives)
19
20         # Plot Hasil
21         stateValues = [current.stateValue]
22         iters = [0]
23         initial_stateValue = current.stateValue
24
25         restart.append(i)
26         res_iter.append(0)
27         res_val.append(0)
28
29         iters_i = 0
30         while True:
31             # generate successors
32             successors = current.generateAllSuccessors(*objectives)
33             if not successors:
34                 break
35
36             # compare value
37             next = max(successors, key=lambda s: s.stateValue)
38             if next.stateValue < current.stateValue:
39                 res_iter[i] = iters_i
40                 res_val[i] = current.stateValue
41                 iters_i = 0
42                 break
43             current = next
44
45         stateValues.append(current.stateValue)
46         iters.append(iters[-1]+1)
47
48         iters_i += 1
49
50         if current.stateValue == 0:
51             res_iter[i] = iters_i
52             res_val[i] = current.stateValue
53             found = True
54             break
55
56         if best is None or current.stateValue > best.stateValue:
57             best = current
58
59     # Plot Hasil
60     plotting_hasil = {
61         'x_label': 'Iterasi',
62         'y_label': 'State Value',
63         'x_data': iters,
64         'y_data': stateValues
65     },
66     {
67         'x_label': 'Restart',
68         'y_label': 'State Value',
69         'x_data': restart,
70         'y_data': res_val
71     },
72     {
63         'x_label': 'Restart',
74         'y_label': 'Jumlah Iterasi',
75         'x_data': restart,
76         'y_data': res_iter
77     }
78
79     stats = {
80         'initial_stateValue': initial_stateValue,
81         'final_stateValue': best.stateValue,
82         'restarts': len(restart),
83         'iter_restart': res_iter,
84         'plot_graph': plotting_hasil
85     }
86
87     return best.copy(), stats

```

1. Mulai dari state copy, lalu langsung di-random satu kali dulu pakai initializeRandomSuccessor
2. Setiap restart, simpan indeks restart ke list restart.
3. Masuk ke loop hill climbing lokal: Ambil semua successors, pilih yang paling tinggi value.
4. Kalau tidak lebih baik break, dan anggap stuck.
5. Kalau lebih baik makageser current ke situ, tambah iterasi, log untuk grafik.
6. Kalau stateValue = 0 langsung break semua, karena berarti sudah optimal.
7. Setiap restart dicatat, untuk di plotting.

2.4.2 Algoritma Simulated Annealing

Algoritma	Garis Besar Cara Kerja
<pre> from src.model.state import State from src.algorithms.local_search import LocalSearch from typing import Tuple, Dict, Any import random import math # Import math untuk exponential function class SA(LocalSearch): def __init__(self, initialT: float, coolingRate: float, stopT: float): self.initialT = initialT # suhu awal self.coolingRate = coolingRate # laju pendinginan self.stopT = stopT # suhu yang menghentikan algoritma def search(self, state: State, objectives: Dict[str, float], stateValue: float, objValue: float): start_time = time.time() currT = self.initialT currState = state currStateValue = currState.stateValue(objectives) initial_stateValue = currState.stateValue iterations = 0 stuck_freq = 0 # plotting hasil all_stateValues = [initial_stateValue] all_probabilities = [1] iterasi = [iterations] print("Memulai Simulated Annealing...") print(f"Initial Temperature: {self.initialT}") print(f"Initial State Value: {initial_stateValue}") while currT > self.stopT: # kalo masih di atas suhu berhenti successor = currState.generateRandomSuccessor(objectives) # cari successor (random) deltaE = successor.stateValue - currState.stateValue # hitung delta E = E_successor - E_curr if deltaE < 0: # Successor lebih baik probability = 1.0 # Delta diterima 100% currState = successor # langsung pindah ke successor else: # Successor lebih buruk, pindah dengan probabilitas tertentu probability = self.acceptance_probability(deltaE, currT) accept = random.uniform(0, 1) # a random float antara 0 dan 1 stuck_freq += 1 if accept < probability: # kalo _deltaE nya ada currState = successor # Successor random # Update Temperature currT = self.coolingRate iterations += 1 # Update Plotting Hasil all_stateValues.append(currState.stateValue) all_probabilities.append(probability) iterasi.append(iterations) # Progress indicator setiap 100 iterasi if iterations % 100 == 0: print(f"Iterasi {iterations}, Temperature: {currT}, State Value: {currState.stateValue}") end_time = time.time() execution_time = end_time - start_time print("Simulated Annealing selesai!") print(f"Final Temperature: {currT}") print(f"Final State Value: {currState.stateValue}") print(f"Final Iterasi: {iterations}") print(f"Delta (kecilnya: execution_time: 0.01 detik)") # plotting hasil plotting_hasil = { 'State Value': { 'x_label': 'Iterasi', 'y_label': 'State Value', 'x_data': iterasi, 'y_data': all_stateValues }, 'Acceptance Probability': { 'x_label': 'Iterasi', 'y_label': 'Acceptance Probability', 'x_data': iterasi, 'y_data': all_probabilities } } # Return state dan statistik stats = { 'algorithm_name': 'Simulated Annealing', 'initial_temp': self.initialT, 'cooling_rate': self.coolingRate, 'stop_temp': self.stopT, 'execution_time': execution_time, 'iterations': iterations, 'stuck_freq': stuck_freq, 'initial_stateValue': initial_stateValue, 'final_stateValue': currState.stateValue, 'plot_graph': plotting_hasil } return currState, stats def acceptance_probability(self, deltaE: float, temperature: float) -> float: # cek ini 0.0000 # baru agecek if temperature <= 0: return 0.0 # untuk _deltaE < 0, exp(_deltaE/T) memberikan probabilitas 0-1 # Semakin besar _deltaE (semakin buruk), semakin kecil probabilitas # Semakin tinggi temperature, semakin besar probabilitas return math.exp(deltaE / temperature) def setInitialTemperature(self, initialT: float): self.initialT = initialT def setCoolingRate(self, coolingRate: float): self.coolingRate = coolingRate def setStopTemperature(self, stopT: float): self.stopT = stopT </pre>	Secara keseluruhan ini adalah class SA yang mengextends class LocalSearch, pada Simulated Annealing memiliki atribut dan method dasar yang digunakan untuk method utama (search), seperti atribut Temperatur inisial, rata rata penurunan temperatur, dan temperatur untuk menghentikan proses, dan juga method getter dan setter atribut diatas. Method search yang diextend dari Local Search. awalnya butuh parameter State dan objectives sehingga menghasilkan Tuple yang berisi State, dengan Dictionary (yang berisi informasi output dari proses SA yang dijalankan). Awal fungsi dipanggil, otomatis time akan dijalankan, dan kemudian menginisiasi value sekarang dan mengassign Temperature yang dipunyai nya. Setelah semuanya yang diperlukan ada. masuk kedalam inti program SA. Selama T sekarang masih lebih besar daripada T stop. Maka akan melakukan generate random successor. Setelah Random State dipilih, maka akan dihitung _deltaE nya yaitu Successor Value - State Sekarang Value. kalau _deltaE nya (+) atau Random State Value lebih besar, maka otomatis pindah. Tapi kalo tidak, maka akan melalui probabilitas perhitungan, yaitu $e^{-\delta E / T}$ dan dilakukan perbandingan dengan Random Function Constant, apabila probabilitas lebih besar dari pada Random Function Constant maka state berpindah, kalau tidak, state

	tidak berpindah. While loop ini akan diakhiri dengan seiring $T = \text{Cooling Rate}$, sampai $T \leq \text{Stop T}$. Akhirnya akan di return dalam Tuple, berisi State Final dan OutputServices nya.
--	--

2.4.3 Algoritma Genetik

Algoritma	Garis Besar Cara Kerja
<pre>def search(self, state:State, "objectives":Callable, conf_crossover=None) -> Tuple[State, Dict[str, Any]]: self.state = state if self.jal_parent % 2 != 0: raise ValueError("N harus kelipatan 2.") self.initialize_availability_for_mutasi() self.initialize_conf_crossover(conf_crossover) start_time = time.time() # 1. Inisialisasi Parent # print("\n[*] Inisialisasi Parent") selected_parent = self._initialize_parent("objectives") min_statevalue = min(s.statevalue for s in selected_parent) max_statevalue = max(s.statevalue for s in selected_parent) avg_statevalue = sum(s.statevalue for s in selected_parent)/self.jal_parent iterasi = 0 min_statevalue = min(s.statevalue for s in selected_parent) best_state = s for s in selected_parent if s.statevalue == max(s.statevalue for s in selected_parent) found = False for i in range(self.jal_iterasi): # print(f"===== Iterasi {i} =====") # 2. Menhitung fitness function # try: # print(f"[*] Fitness function\n (fitness_function -> ", end="") # except: # print(f"[*] Fitness function\n [] -> ", end="") adaptive_scaler = abs(min_statevalue) + 1 fitness_function = [] for s in enumerate(selected_parent): fitness_function.append(adaptive_scaler * s.statevalue) # print(f"[*] Fitness function") # print(f"[*] Adaptive Scaler (1): (adaptive_scaler)") # print(f"[*] Generation State value {i}\n {(s.statevalue for s in selected_parent)}") # 3. Selection # print(f"[*] Selection (1)") selected_parent = random.choices(selected_parent, weights=fitness_function, k=self.jal_parent) # 4. Crossover # print(f"[*] Crossover (1)") for j in range(0, self.jal_parent, 2): p1 = selected_parent[j] p2 = selected_parent[j+1] # print(f"[*] Crossover (1): ({p1}, {p2})") c1, c2 = self._crossover(p1, p2, "objectives") # Komparasi fitness urutan_state = sorted([p1, p2, c1, c2], key=lambda x: x.statevalue, reverse=True) good_state1 = urutan_state[0] good_state2 = urutan_state[1] # 5. Mutation # print(f"[*] Mutation (1): ({p1}, {p2})") good_state1 = self._mutation(good_state1, "objectives") good_state2 = self._mutation(good_state2, "objectives") # cek interpretasi solusi if good_state1.statevalue > best_state.statevalue: best_state = good_state1 if good_state2.statevalue > best_state.statevalue: best_state = good_state2 selected_parent[j] = good_state1 selected_parent[j+1] = good_state2 if good_state1.statevalue == 0 or good_state2.statevalue == 0: found = True; break # 5. Evaluasi min_statevalue min_statevalue = min(min_statevalue, min(s.statevalue for s in selected_parent)) # Plot hasil max_statevalue.append(max(s.statevalue for s in selected_parent)) avg_statevalue.append(sum(s.statevalue for s in selected_parent)/self.jal_parent) iterasi.append(iterasi+1) if found: break end_time = time.time() execution_time = end_time - start_time # plotting hasil plotting_hasil = { "Maximum State Values": { "x_label": "Iterasi", "y_label": "State Value", "x_data": iterasi, "y_data": max_statevalue }, "Average State Values": { "x_label": "Iterasi", "y_label": "State Value", "x_data": iterasi, "y_data": avg_statevalue } } stats = { "Algorithm name": "Genetic Algorithm", "Iterations": iterasi-1, "Initial statevalue": max_statevalue[0], "Final statevalue": best_state.statevalue, "Execution time": execution_time, "Plot graph": plotting_hasil } print("===== Done =====\n") return best_state, stats</pre>	Ini adalah fungsi search yang akan melakukan langkah-langkah berikut untuk mencari solusi: 1. Inisialisasi parent sejumlah yang diinginkan. Inisialisasi menghasilkan parent yang random menggunakan randomSuccessor dari state awal. 2. Melakukan iterasi siklus generasi yang terdiri atas: 2.1 Perhitungan fitness function yang merupakan penjumlahan adaptive scaler dengan state value (state value bernilai negatif). adaptive scaler merupakan state value terburuk yang pernah ditemui. 2.1 Selection: pemilihan anakan berdasarkan roulette wheel dengan probabilitas diambil dari fitness function. 2.2 Crossover: akan dijelaskan 2.3 Mutation: akan dijelaskan 3. Iterasi dihentikan apabila telah ditemukan solusi (state value = 0) atau telah mencapai maksimum iterasi yang diinginkan. 4. Hasil diambil dari individu terbaik dari seluruh populasi yang ada.

<pre> def _crossover(self, stateA:State, stateB:State, *objectives:Callable) -> Tuple[State, State]: """Melakukan crossover""" state_A = stateA.copy() state_B = stateB.copy() # 1. Extract (matkul -> slots) matkul_slots_A = state_A.mtkulslots matkul_slots_B = state_B.mtkulslots # 2. Crossover random_crossover_selector = random.randint(0, 1) for kode_matkul, list_crossoverability in self.conf.crossover.items(): for i, crossoverable in enumerate(list_crossoverability): if ((random_crossover_selector==0 and crossoverable) or (random_crossover_selector==1 and not crossoverable)): state_A.swapMK(matkul_slots_A[kode_matkul][i], matkul_slots_B[kode_matkul][i]) state_B.swapMK(matkul_slots_A[kode_matkul][i], matkul_slots_B[kode_matkul][i]) # 3. Update stateValue state_A.stateValue = state_A.countStateValue(*objectives) state_B.stateValue = state_B.countStateValue(*objectives) return state_A, state_B </pre>	2.2 Crossover Crossover merupakan proses pertukaran dua buah assignment (slot yang terisi) mata kuliah yang sama dari dua individu yang berbeda. Proses ini serupa dengan melakukan re-assignment dari sebuah mata kuliah pada slot tertentu ke slot lain, yaitu slot yang pada individu pasangannya berisi mata kuliah yang sama dengan pasangannya. Re-assignment dilakukan dengan swapMK().
<pre> def _mutation(self, state:State, *objectives:Callable) -> State: """Melakukan Mutation""" new_state = state.generateRandomSuccessor(*objectives) if new_state.stateValue >= state.stateValue: return new_state else: return state.copy() </pre>	2.3 Mutation Strategi mutasi yang diberlakukan adalah dengan menggunakan random successor dari state tersebut. Agar selalu menghasilkan generasi yang membaik, mutasi digagalkan apabila random successor yang terpilih tidak lebih baik.

BAB III

EKSPERIMEN DAN ANALISIS

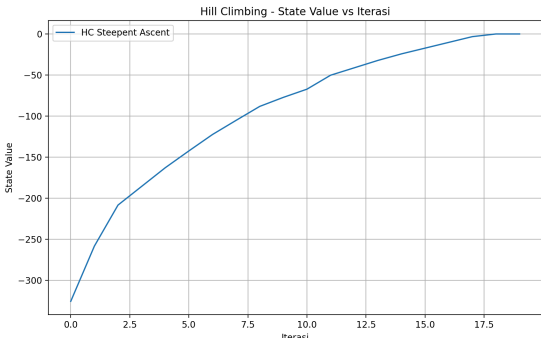
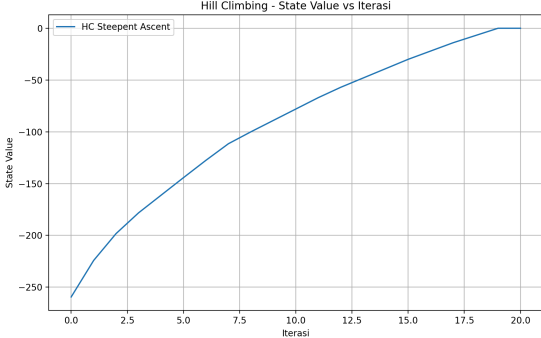
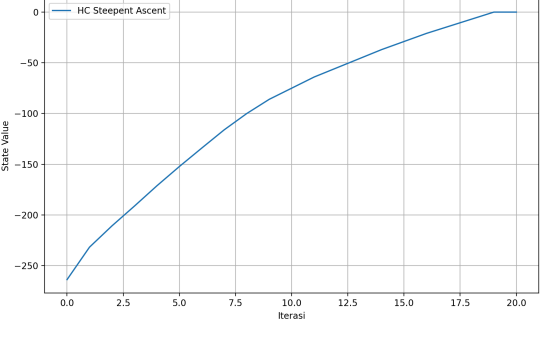
3.1 Eksperimen

3.1.1 Hill Climbing

3.1.1.1 Steepest Ascent

pengujian ke-i	banyak iterasi	Hasil
1	19	INFORMASI ALGORITMA ----- Algoritma : Hill Climbing Variant : steepest Waktu Eksekusi : 31.881 detik Jumlah Iterasi : 19 State Value Awal : -325.75 State Value Akhir : 0.00 Peningkatan : +325.75 NILAI FUNGSI OBJEKTIF ----- State Value Akhir : 0.00
2	20	INFORMASI ALGORITMA ----- Algoritma : Hill Climbing Variant : steepest Waktu Eksekusi : 32.040 detik Jumlah Iterasi : 20 State Value Awal : -259.50 State Value Akhir : 0.00 Peningkatan : +259.50 NILAI FUNGSI OBJEKTIF ----- State Value Akhir : 0.00
3	20	INFORMASI ALGORITMA ----- Algoritma : Hill Climbing Variant : steepest Waktu Eksekusi : 30.986 detik Jumlah Iterasi : 20 State Value Awal : -263.75 State Value Akhir : 0.00 Peningkatan : +263.75 NILAI FUNGSI OBJEKTIF ----- State Value Akhir : 0.00

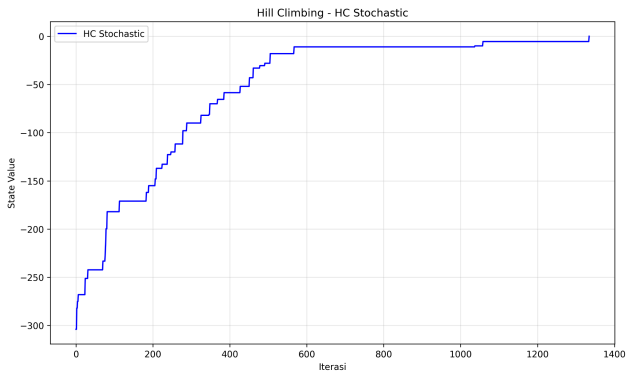
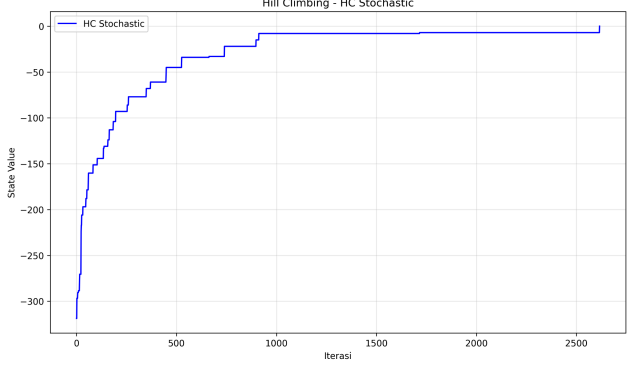
pengujian ke-i	banyak iterasi	Hasil
----------------	----------------	-------

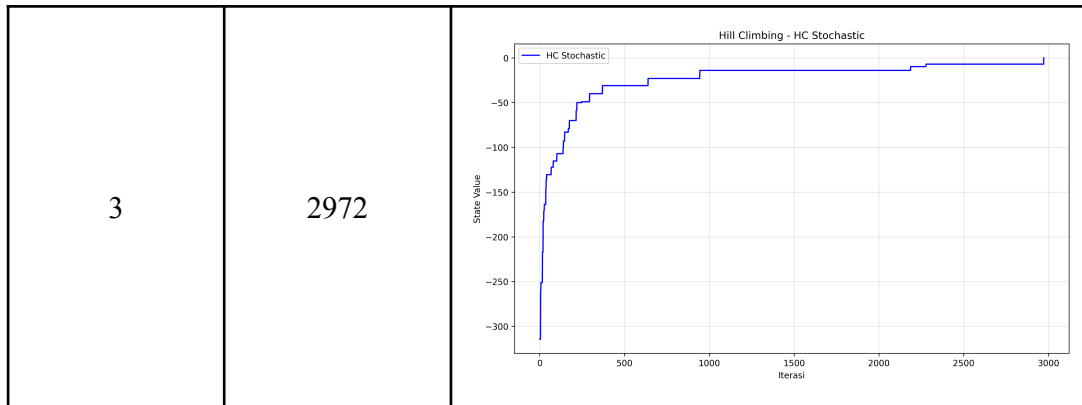
1	19	 The graph shows the state value starting at -300 at iteration 0.0 and rising to 0.0 at iteration 19.0. The curve is smooth and shows a slight dip around iteration 10.0.
2	20	 The graph shows the state value starting at -250 at iteration 0.0 and rising to 0.0 at iteration 20.0. The curve is smooth and shows a slight dip around iteration 10.0.
3	20	 The graph shows the state value starting at -250 at iteration 0.0 and rising to 0.0 at iteration 20.0. The curve is smooth and shows a slight dip around iteration 10.0.

3.1.1.2 Stochastic

pengujian ke-i	banyak iterasi	Hasil
1	1334	INFORMASI ALGORITMA ----- Algoritma : Hill Climbing Variant : stochastic Max Iteration : 1000000 Waktu Eksekusi : 0.215 detik Jumlah Iterasi : 1,334 State Value Awal : -304.00 State Value Akhir : 0.00 Peningkatan : +304.00 NILAI FUNGSI OBJEKTIF ----- State Value Akhir : 0.00

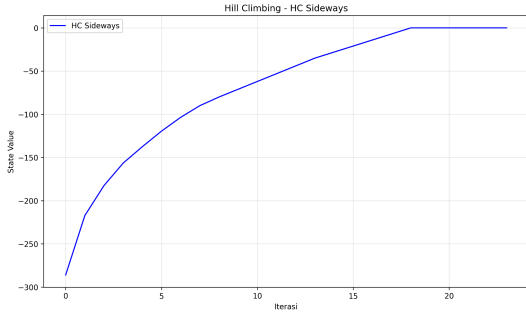
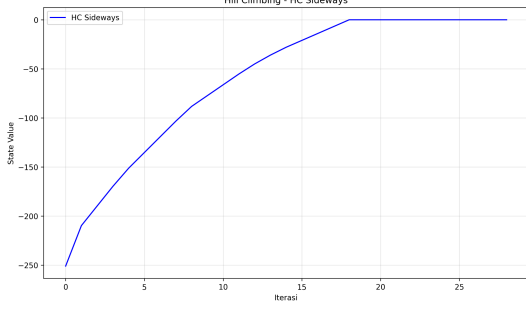
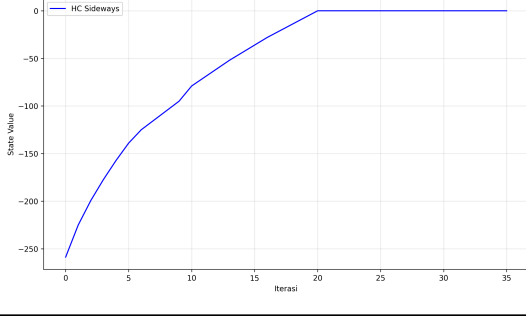
2	2616	INFORMASI ALGORITMA ----- Algoritma : Hill Climbing Variant : stochastic Max Iteration : 1000000 Waktu Eksekusi : 0.411 detik Jumlah Iterasi : 2,616 State Value Awal : -318.75 State Value Akhir : 0.00 Peningkatan : +318.75 NILAI FUNGSI OBJEKTIF ----- State Value Akhir : 0.00
3	2972	INFORMASI ALGORITMA ----- Algoritma : Hill Climbing Variant : stochastic Max Iteration : 1000000 Waktu Eksekusi : 0.482 detik Jumlah Iterasi : 2,972 State Value Awal : -314.50 State Value Akhir : 0.00 Peningkatan : +314.50 NILAI FUNGSI OBJEKTIF ----- State Value Akhir : 0.00

pengujian ke-i	banyak iterasi	Hasil
1	1334	
2	2616	



3.1.1.3 Sideways Move

pengujian ke-i	max sideways	banyak iterasi	Hasil
1	5	23	INFORMASI ALGORITMA ----- Algoritma : Hill Climbing Variant : sideways Max Sideways : 5 Waktu Eksekusi : 38.184 detik Jumlah Iterasi : 23 State Value Awal : -286.00 State Value Akhir : 0.00 Peningkatan : +286.00 NILAI FUNGSI OBJEKTIF ----- State Value Akhir : 0.00
2	10	28	INFORMASI ALGORITMA ----- Algoritma : Hill Climbing Variant : sideways Max Sideways : 10 Waktu Eksekusi : 46.187 detik Jumlah Iterasi : 28 State Value Awal : -251.00 State Value Akhir : 0.00 Peningkatan : +251.00 NILAI FUNGSI OBJEKTIF ----- State Value Akhir : 0.00
3	15	35	INFORMASI ALGORITMA ----- Algoritma : Hill Climbing Variant : sideways Max Sideways : 15 Waktu Eksekusi : 61.282 detik Jumlah Iterasi : 35 State Value Awal : -258.75 State Value Akhir : 0.00 Peningkatan : +258.75 NILAI FUNGSI OBJEKTIF ----- State Value Akhir : 0.00

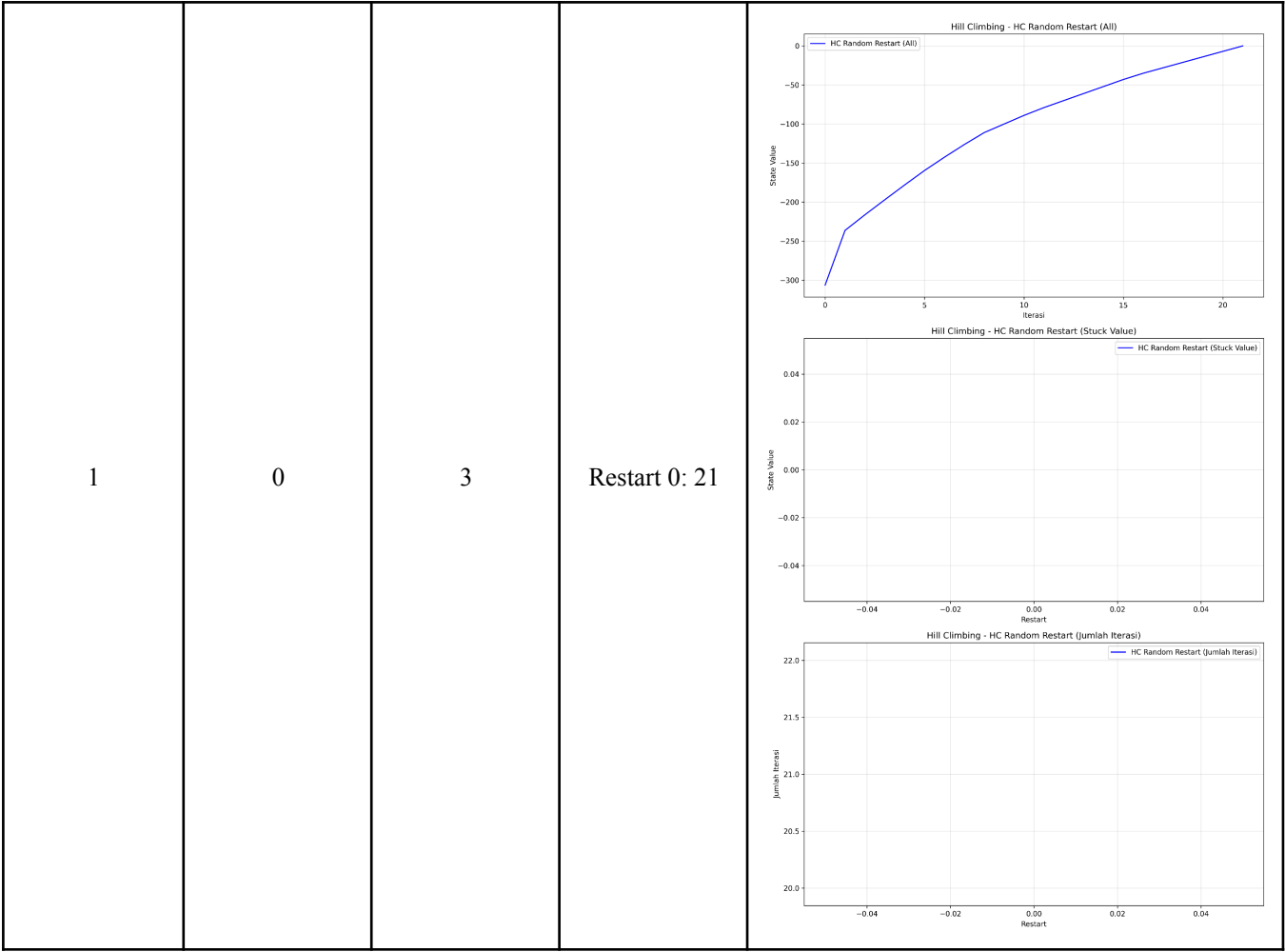
pengujian ke-i	max sideways	banyak iterasi	Hasil
1	5	23	
2	10	28	
3	15	35	

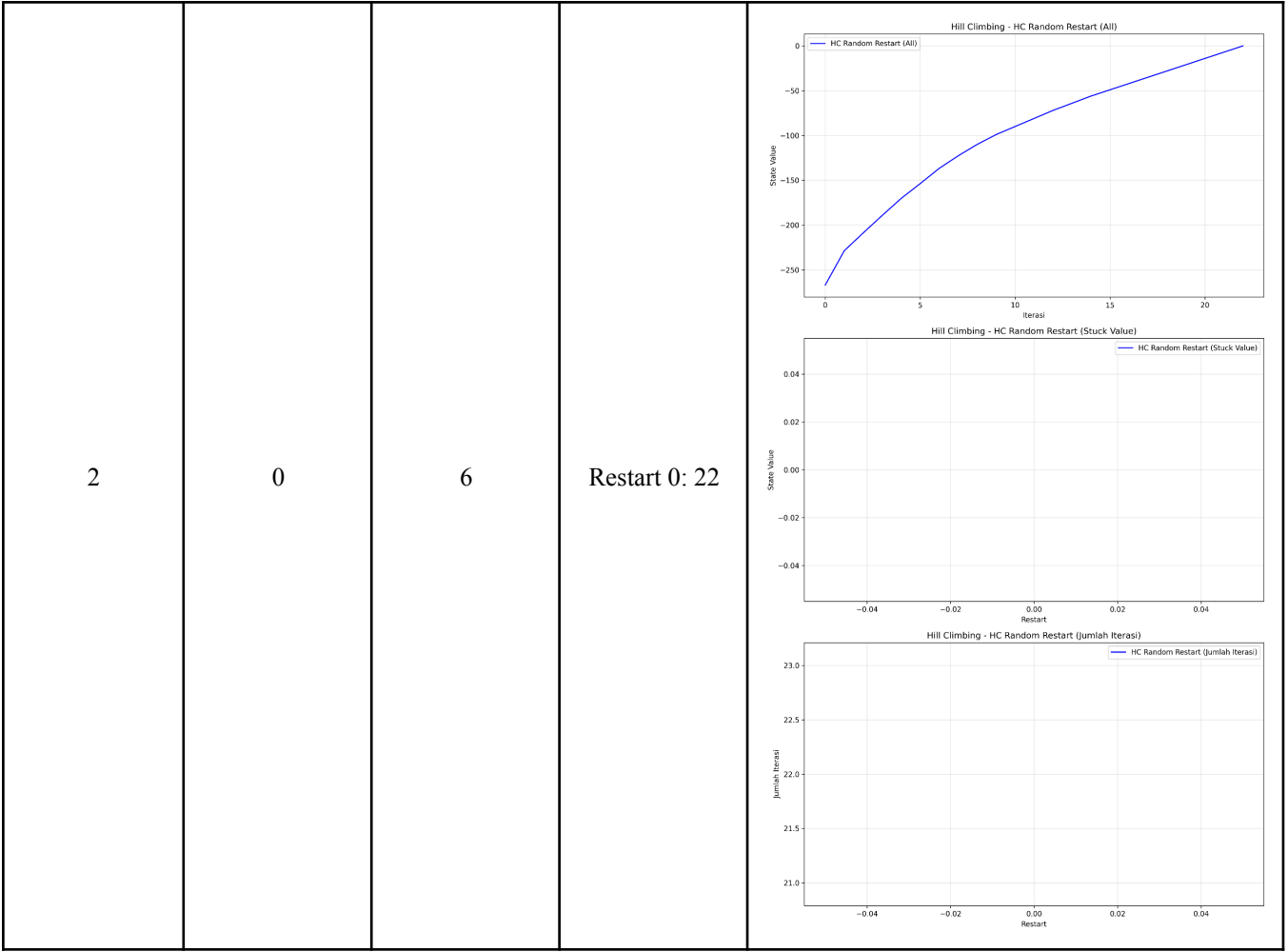
3.1.1.4 Random Restart

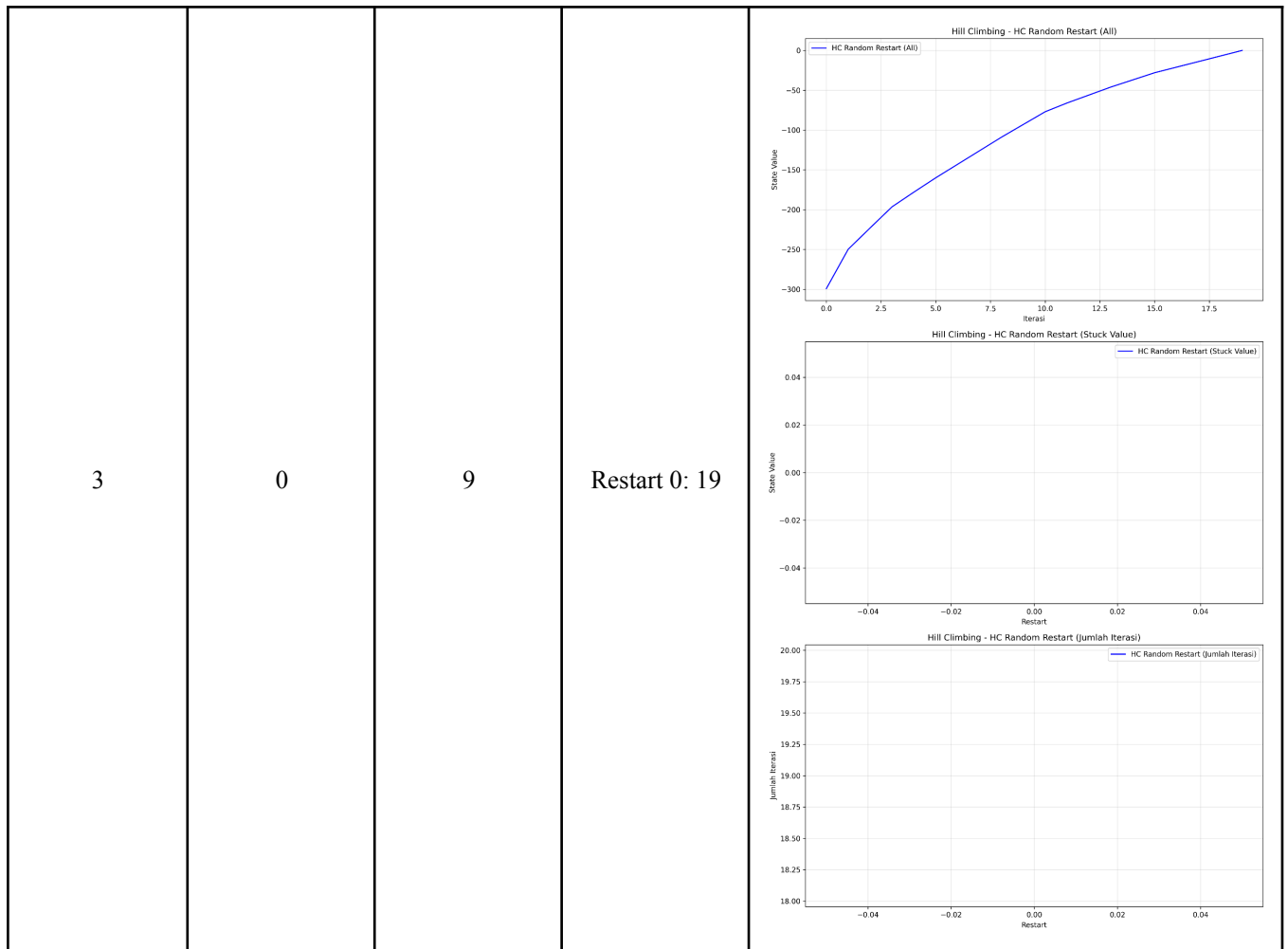
pengujian ke-i	banyak restart	max restart	banyak iterasi	Hasil
1	0	3	Restart 0: 21	INFORMASI ALGORITMA ----- Algoritma : Hill Climbing Variant : random_restart Max Restart : 3 Waktu Eksekusi : 35.188 detik State Value Awal : -306.25 State Value Akhir : 0.00 Peningkatan : +306.25 Jumlah Restart : 0 Iterasi Restart ke-0: 21

				NILAI FUNGSI OBJEKTIF ----- State Value Akhir : 0.00
2	0	6	Restart 0: 22	INFORMASI ALGORITMA ----- Algoritma : Hill Climbing Variant : random_restart Max Restart : 6 Waktu Eksekusi : 38.543 detik State Value Awal : -267.00 State Value Akhir : 0.00 Peningkatan : +267.00 Jumlah Restart : 0 Iterasi Restart ke-0: 22 NILAI FUNGSI OBJEKTIF ----- State Value Akhir : 0.00
3	0	9	Restart 0: 19	INFORMASI ALGORITMA ----- Algoritma : Hill Climbing Variant : random_restart Max Restart : 9 Waktu Eksekusi : 32.834 detik State Value Awal : -299.25 State Value Akhir : 0.00 Peningkatan : +299.25 Jumlah Restart : 0 Iterasi Restart ke-0: 19 NILAI FUNGSI OBJEKTIF ----- State Value Akhir : 0.00

pengujian ke-i	banyak restart	max restart	banyak iterasi	Hasil
----------------	----------------	-------------	----------------	-------







3.1.2 Simulated Annealing

Untuk Frekuensi Stuck berada di Output Parameter & Hasil

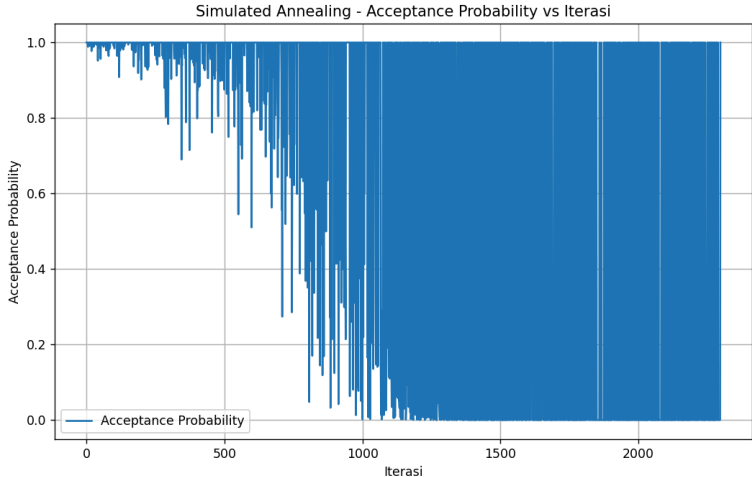
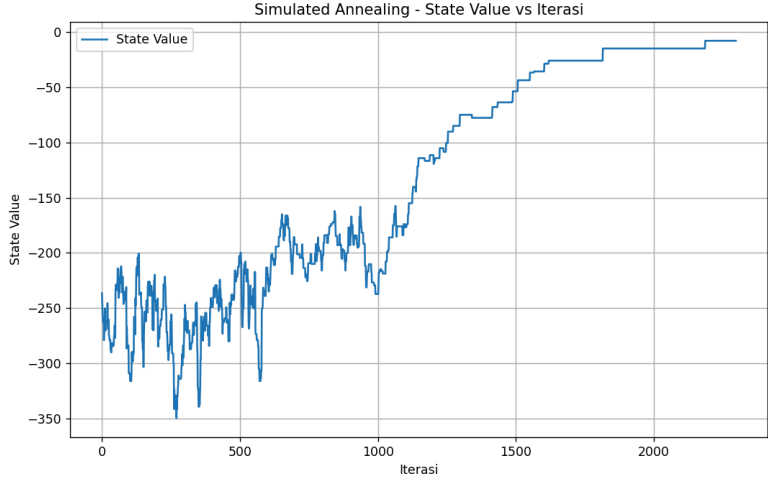
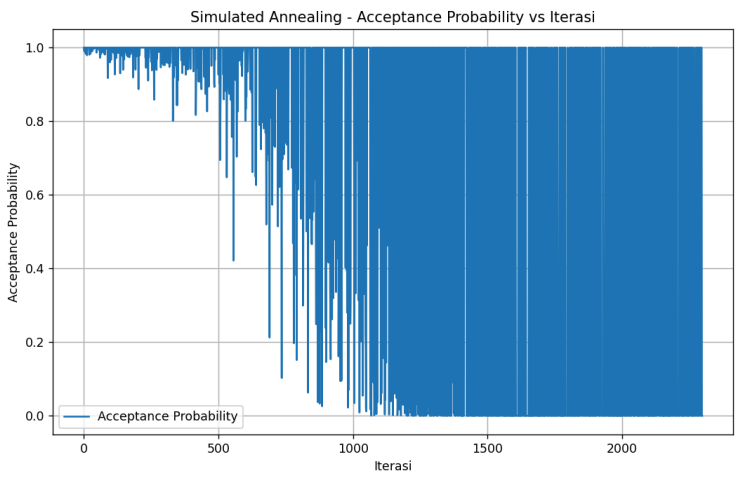
initialT	coolingRate	stopT	penguji an ke-i	Hasil
1000	0.995	0.01	1	<pre> +-----+ PARAMETER & HASIL -----+-----+ Algoritma : Simulated Annealing Initial Temperature : 1000.00 Cooling Rate : 0.9950 Stop Temperature : 0.0100 Waktu Eksekusi : 2.242 detik Jumlah Iterasi : 2,297 State Value Awal : -284.00 State Value Akhir : 0.00 Peningkatan : +284.00 Stuck Frequency : 2047 -----+-----+ </pre>

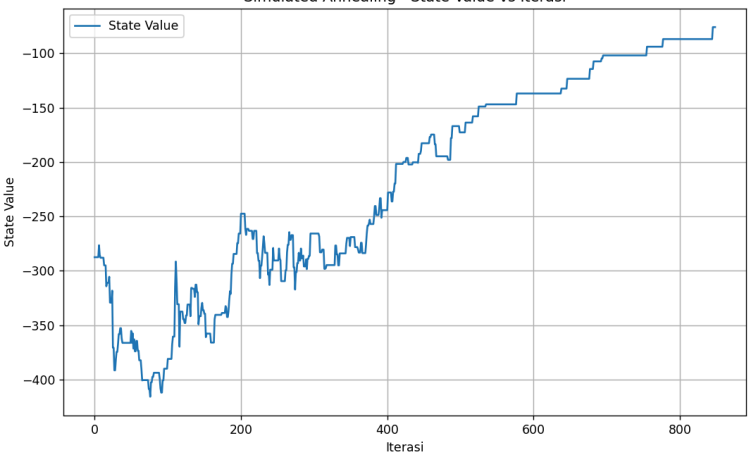
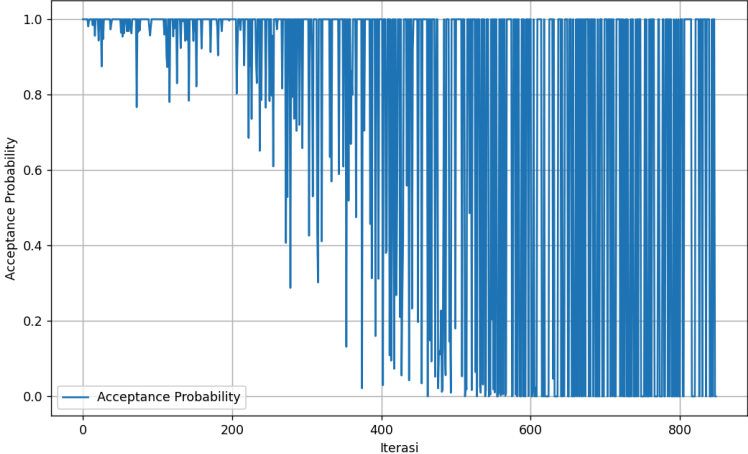
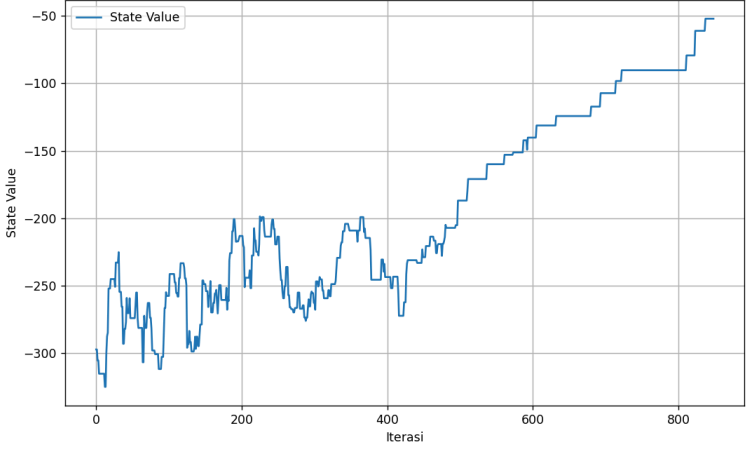
				+-----+
1000	0.995	0.01	2	+-----+ PARAMETER & HASIL +-----+ Algoritma : Simulated Annealing Initial Temperature : 1000.00 Cooling Rate : 0.9950 Stop Temperature : 0.0100 Waktu Eksekusi : 2.205 detik Jumlah Iterasi : 2,297 State Value Awal : -317.25 State Value Akhir : -25.00 Peningkatan : +292.25 Stuck Frequency : 2050 +-----+
1000	0.995	0.01	3	+-----+ PARAMETER & HASIL +-----+ Algoritma : Simulated Annealing Initial Temperature : 1000.00 Cooling Rate : 0.9950 Stop Temperature : 0.0100 Waktu Eksekusi : 2.068 detik Jumlah Iterasi : 2,297 State Value Awal : -236.50 State Value Akhir : -8.00 Peningkatan : +228.50 Stuck Frequency : 2036 +-----+
500	0.99	0.1	1	+-----+ PARAMETER & HASIL +-----+ Algoritma : Simulated Annealing Initial Temperature : 500.00 Cooling Rate : 0.9900 Stop Temperature : 0.1000 Waktu Eksekusi : 1.073 detik Jumlah Iterasi : 848 State Value Awal : -287.50 State Value Akhir : -76.00 Peningkatan : +211.50 Stuck Frequency : 731 +-----+
500	0.99	0.1	2	+-----+ PARAMETER & HASIL +-----+ Algoritma : Simulated Annealing Initial Temperature : 500.00 Cooling Rate : 0.9900 Stop Temperature : 0.1000 Waktu Eksekusi : 0.921 detik Jumlah Iterasi : 848 State Value Awal : -297.50 State Value Akhir : -52.00 Peningkatan : +245.50 Stuck Frequency : 731 +-----+
500	0.99	0.1	3	+-----+ PARAMETER & HASIL +-----+ Algoritma : Simulated Annealing Initial Temperature : 500.00 Cooling Rate : 0.9900 Stop Temperature : 0.1000 Waktu Eksekusi : 0.816 detik +-----+

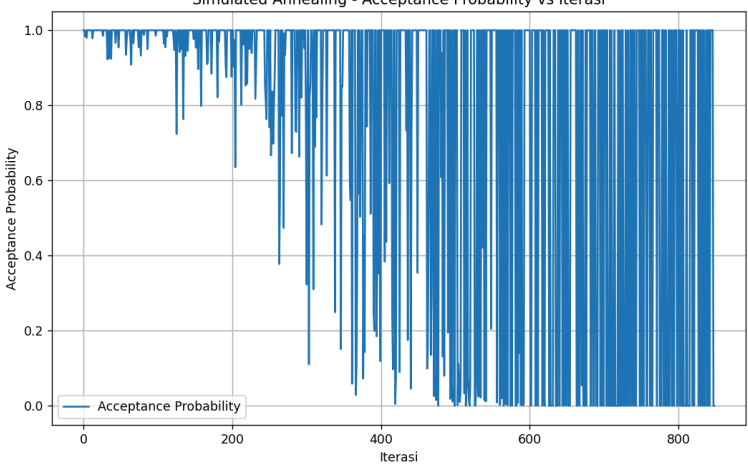
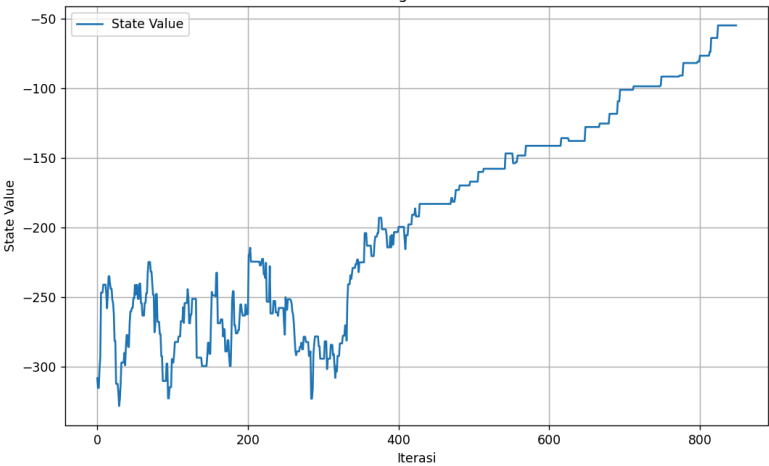
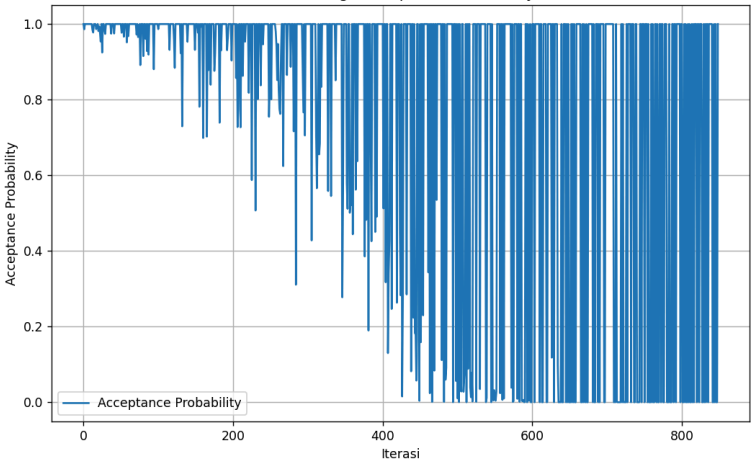
				<pre> Jumlah Iterasi : 848 State Value Awal : -308.50 State Value Akhir : -55.00 Peningkatan : +253.50 Stuck Frequency : 719 +-----+ </pre>
100	0.95	1	1	<pre> +-----+ PARAMETER & HASIL +-----+ Algoritma : Simulated Annealing Initial Temperature : 100.00 Cooling Rate : 0.9500 Stop Temperature : 1.0000 Waktu Eksekusi : 0.096 detik Jumlah Iterasi : 90 State Value Awal : -299.25 State Value Akhir : -201.75 Peningkatan : +97.50 Stuck Frequency : 74 +-----+ </pre>
100	0.95	1	2	<pre> +-----+ PARAMETER & HASIL +-----+ Algoritma : Simulated Annealing Initial Temperature : 100.00 Cooling Rate : 0.9500 Stop Temperature : 1.0000 Waktu Eksekusi : 0.100 detik Jumlah Iterasi : 90 State Value Awal : -247.75 State Value Akhir : -212.00 Peningkatan : +35.75 Stuck Frequency : 71 +-----+ </pre>
100	0.95	1	3	<pre> +-----+ PARAMETER & HASIL +-----+ Algoritma : Simulated Annealing Initial Temperature : 100.00 Cooling Rate : 0.9500 Stop Temperature : 1.0000 Waktu Eksekusi : 0.099 detik Jumlah Iterasi : 90 State Value Awal : -265.50 State Value Akhir : -179.75 Peningkatan : +85.75 Stuck Frequency : 70 +-----+ </pre>

initalT	cooling Rate	stopT	penguji an ke-i	Hasil
---------	--------------	-------	-----------------	-------

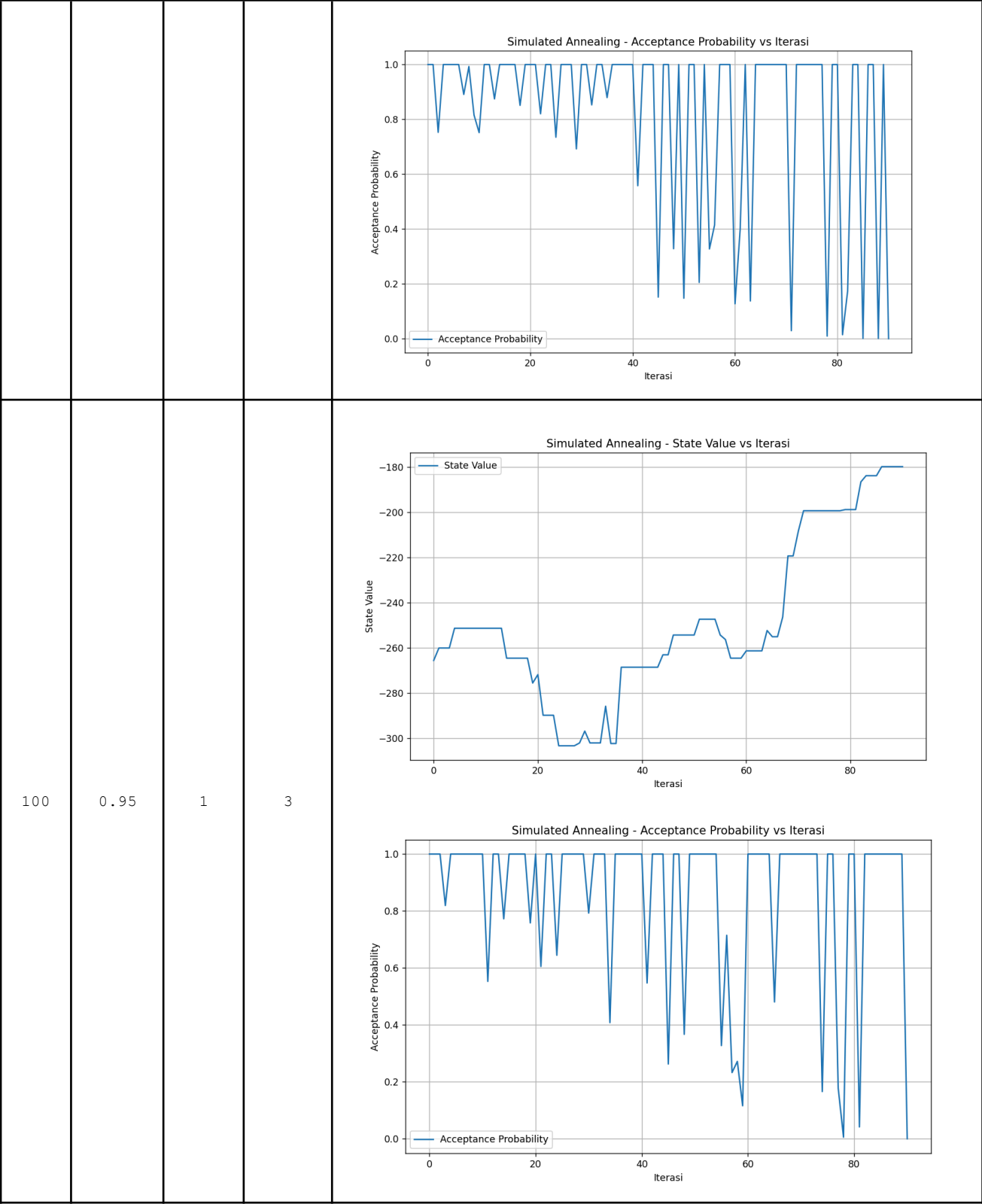
1000	0.995	0.01	1	Simulated Annealing - State Value vs Iterasi Simulated Annealing - Acceptance Probability vs Iterasi
1000	0.995	0.01	2	Simulated Annealing - State Value vs Iterasi

				Simulated Annealing - Acceptance Probability vs Iterasi 
1000	0.995	0.01	3	Simulated Annealing - State Value vs Iterasi  Simulated Annealing - Acceptance Probability vs Iterasi 

500	0.99	0.1	1	Simulated Annealing - State Value vs Iterasi  Simulated Annealing - Acceptance Probability vs Iterasi 
500	0.99	0.1	2	Simulated Annealing - State Value vs Iterasi 

				Simulated Annealing - Acceptance Probability vs Iterasi 
500	0.99	0.1	3	Simulated Annealing - State Value vs Iterasi  Simulated Annealing - Acceptance Probability vs Iterasi 

100	0.95	1	1	Simulated Annealing - State Value vs Iterasi Simulated Annealing - Acceptance Probability vs Iterasi
100	0.95	1	2	Simulated Annealing - State Value vs Iterasi



3.1.3 Genetic Algorithm

jml_pop	jml_iter	pengujian ke-i	HASIL
8	100	1	INFORMASI ALGORITMA ----- Algoritma : Genetic Algorithm Population Size : 8 Max Iteration : 100 Waktu Eksekusi : 0.330 detik Jumlah Iterasi : 100 State Value Awal : -249.75 State Value Akhir : -92.75 Peningkatan : +157.00 NILAI FUNGSI OBJEKTIF ----- State Value Akhir : -92.75
8	100	2	INFORMASI ALGORITMA ----- Algoritma : Genetic Algorithm Population Size : 8 Max Iteration : 100 Waktu Eksekusi : 0.314 detik Jumlah Iterasi : 100 State Value Awal : -257.75 State Value Akhir : -61.75 Peningkatan : +196.00 NILAI FUNGSI OBJEKTIF ----- State Value Akhir : -61.75
8	100	3	INFORMASI ALGORITMA ----- Algoritma : Genetic Algorithm Population Size : 8 Max Iteration : 100 Waktu Eksekusi : 0.332 detik Jumlah Iterasi : 100 State Value Awal : -223.25 State Value Akhir : -92.00 Peningkatan : +131.25 NILAI FUNGSI OBJEKTIF ----- State Value Akhir : -92.00
8	1000	1	INFORMASI ALGORITMA ----- Algoritma : Genetic Algorithm Population Size : 8 Max Iteration : 1,000 Waktu Eksekusi : 1.712 detik Jumlah Iterasi : 533 State Value Awal : -211.00 State Value Akhir : 0.00 Peningkatan : +211.00 NILAI FUNGSI OBJEKTIF ----- State Value Akhir : 0.00
8	1000	2	INFORMASI ALGORITMA ----- Algoritma : Genetic Algorithm Population Size : 8 Max Iteration : 1,000 Waktu Eksekusi : 1.315 detik Jumlah Iterasi : 398 State Value Awal : -254.50 State Value Akhir : 0.00 Peningkatan : +254.50

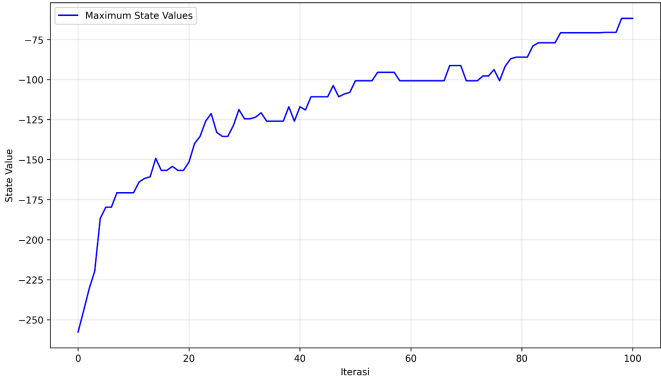
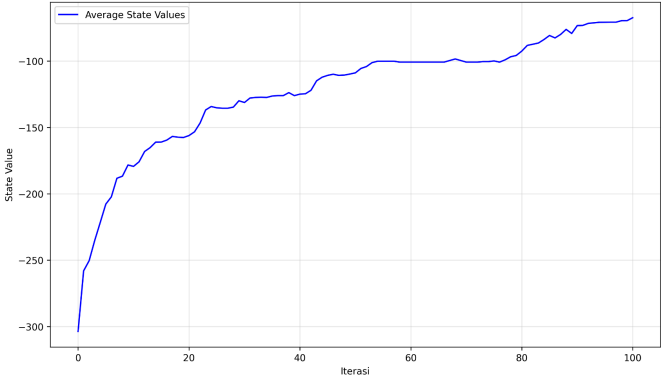
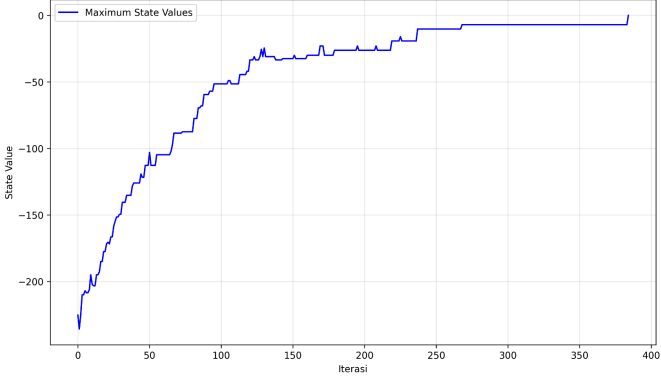
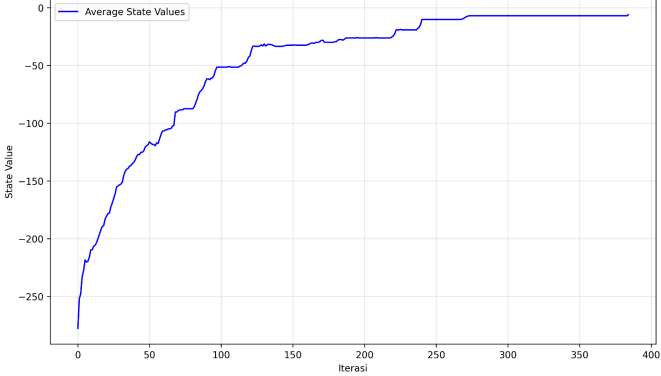
			NILAI FUNGSI OBJEKTIF ----- State Value Akhir : 0.00
8	1000	3	INFORMASI ALGORITMA ----- Algoritma : Genetic Algorithm Population Size : 8 Max Iteration : 1,000 Waktu Eksekusi : 1.288 detik Jumlah Iterasi : 384 State Value Awal : -225.25 State Value Akhir : 0.00 Peningkatan : +225.25 NILAI FUNGSI OBJEKTIF ----- State Value Akhir : 0.00
8	10000	1	INFORMASI ALGORITMA ----- Algoritma : Genetic Algorithm Population Size : 8 Max Iteration : 10,000 Waktu Eksekusi : 1.080 detik Jumlah Iterasi : 344 State Value Awal : -191.50 State Value Akhir : 0.00 Peningkatan : +191.50 NILAI FUNGSI OBJEKTIF ----- State Value Akhir : 0.00
8	10000	2	INFORMASI ALGORITMA ----- Algoritma : Genetic Algorithm Population Size : 8 Max Iteration : 10,000 Waktu Eksekusi : 1.727 detik Jumlah Iterasi : 558 State Value Awal : -230.75 State Value Akhir : 0.00 Peningkatan : +230.75 NILAI FUNGSI OBJEKTIF ----- State Value Akhir : 0.00
8	10000	3	INFORMASI ALGORITMA ----- Algoritma : Genetic Algorithm Population Size : 8 Max Iteration : 10,000 Waktu Eksekusi : 1.934 detik Jumlah Iterasi : 606 State Value Awal : -259.50 State Value Akhir : 0.00 Peningkatan : +259.50 NILAI FUNGSI OBJEKTIF ----- State Value Akhir : 0.00
4	500	1	INFORMASI ALGORITMA ----- Algoritma : Genetic Algorithm Population Size : 4 Max Iteration : 500 Waktu Eksekusi : 0.805 detik Jumlah Iterasi : 500 State Value Awal : -247.75 State Value Akhir : -22.00 Peningkatan : +225.75

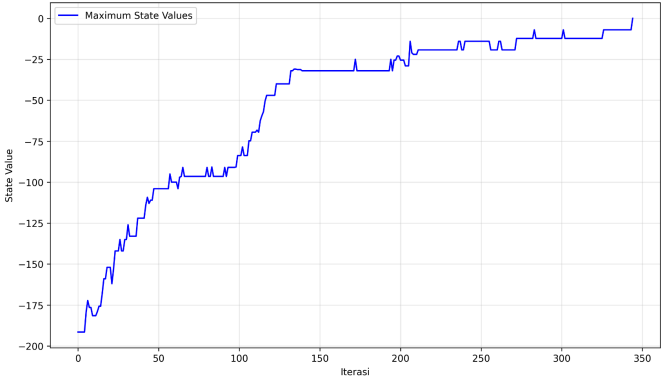
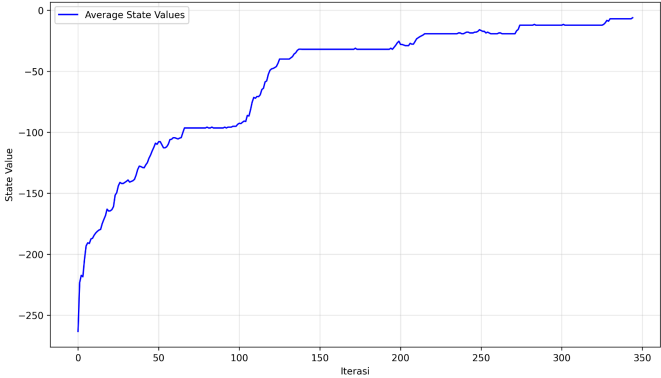
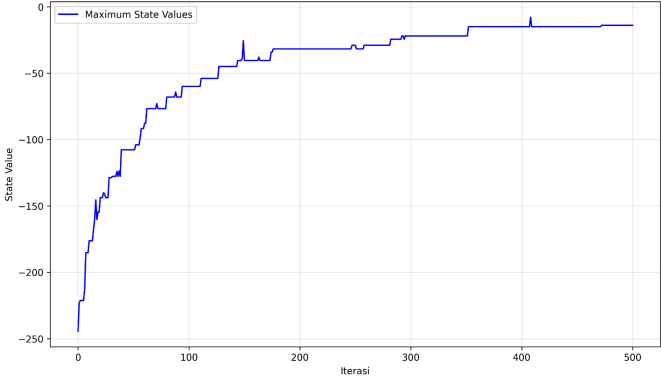
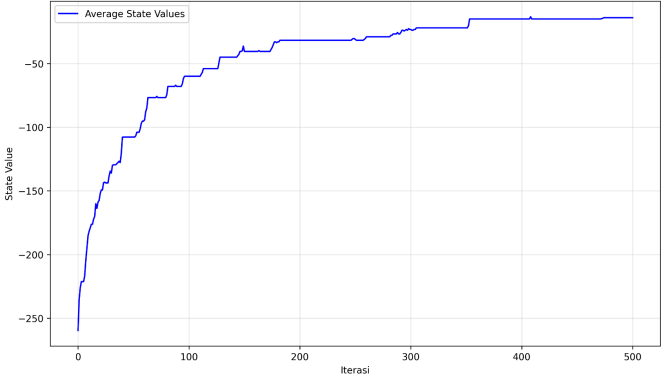
			NILAI FUNGSI OBJEKTIF ----- State Value Akhir : -22.00
4	500	2	INFORMASI ALGORITMA ----- Algoritma : Genetic Algorithm Population Size : 4 Max Iteration : 500 Waktu Eksekusi : 0.826 detik Jumlah Iterasi : 500 State Value Awal : -241.25 State Value Akhir : -8.00 Peningkatan : +233.25 NILAI FUNGSI OBJEKTIF ----- State Value Akhir : -8.00
4	500	3	INFORMASI ALGORITMA ----- Algoritma : Genetic Algorithm Population Size : 4 Max Iteration : 500 Waktu Eksekusi : 0.800 detik Jumlah Iterasi : 500 State Value Awal : -244.25 State Value Akhir : -8.00 Peningkatan : +236.25 NILAI FUNGSI OBJEKTIF ----- State Value Akhir : -8.00
8	500	1	INFORMASI ALGORITMA ----- Algoritma : Genetic Algorithm Population Size : 8 Max Iteration : 500 Waktu Eksekusi : 1.569 detik Jumlah Iterasi : 500 State Value Awal : -213.25 State Value Akhir : -7.00 Peningkatan : +206.25 NILAI FUNGSI OBJEKTIF ----- State Value Akhir : -7.00
8	500	2	INFORMASI ALGORITMA ----- Algoritma : Genetic Algorithm Population Size : 8 Max Iteration : 500 Waktu Eksekusi : 1.562 detik Jumlah Iterasi : 500 State Value Awal : -215.00 State Value Akhir : -7.00 Peningkatan : +208.00 NILAI FUNGSI OBJEKTIF ----- State Value Akhir : -7.00
8	500	3	INFORMASI ALGORITMA ----- Algoritma : Genetic Algorithm Population Size : 8 Max Iteration : 500 Waktu Eksekusi : 1.158 detik Jumlah Iterasi : 366 State Value Awal : -247.50 State Value Akhir : 0.00

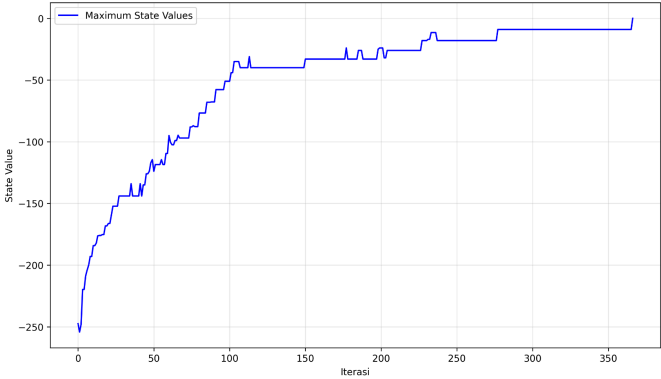
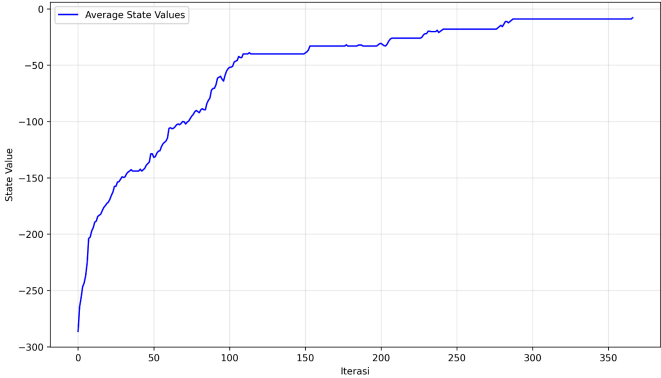
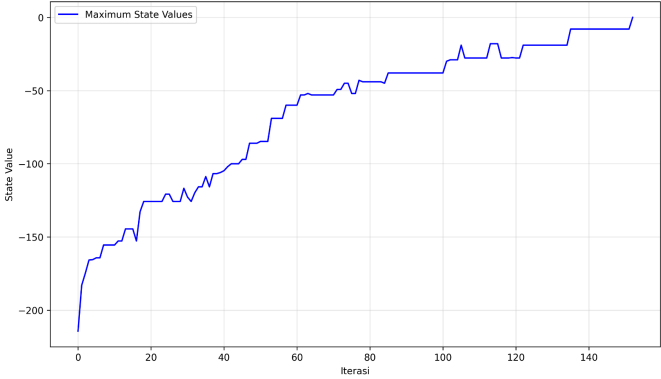
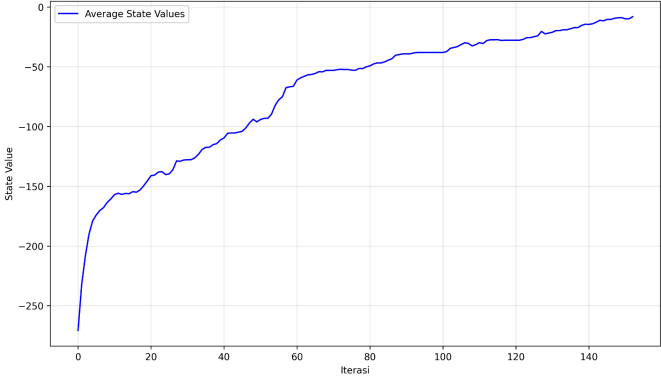
			Peningkatan : +247.50 NILAI FUNGSI OBJEKTIF ----- State Value Akhir : 0.00
12	500	1	INFORMASI ALGORITMA ----- Algoritma : Genetic Algorithm Population Size : 12 Max Iteration : 500 Waktu Eksekusi : 0.902 detik Jumlah Iterasi : 152 State Value Awal : -214.25 State Value Akhir : 0.00 Peningkatan : +214.25 NILAI FUNGSI OBJEKTIF ----- State Value Akhir : 0.00
12	500	2	INFORMASI ALGORITMA ----- Algoritma : Genetic Algorithm Population Size : 12 Max Iteration : 500 Waktu Eksekusi : 2.263 detik Jumlah Iterasi : 463 State Value Awal : -227.75 State Value Akhir : 0.00 Peningkatan : +227.75 NILAI FUNGSI OBJEKTIF ----- State Value Akhir : 0.00
12	500	3	INFORMASI ALGORITMA ----- Algoritma : Genetic Algorithm Population Size : 12 Max Iteration : 500 Waktu Eksekusi : 1.626 detik Jumlah Iterasi : 337 State Value Awal : -220.75 State Value Akhir : 0.00 Peningkatan : +220.75 NILAI FUNGSI OBJEKTIF ----- State Value Akhir : 0.00

Grafik untuk hasil terbaik dari setiap pengulangan pengujian berdasarkan hasil state value akhir dan waktu eksekusi.

jml_pop	jml_iter	pengujian ke-i	HASIL
---------	----------	----------------	-------

8	100	2	Genetic Algorithm - Maximum State Values  Genetic Algorithm - Average State Values 
8	1000	3	Genetic Algorithm - Maximum State Values  Genetic Algorithm - Average State Values 

8	10000	1	Genetic Algorithm - Maximum State Values  Genetic Algorithm - Average State Values 
4	500	3	Genetic Algorithm - Maximum State Values  Genetic Algorithm - Average State Values 

8	500	3	Genetic Algorithm - Maximum State Values  Genetic Algorithm - Average State Values 
12	500	1	Genetic Algorithm - Maximum State Values  Genetic Algorithm - Average State Values 

3.2 Analisis

Berdasarkan hasil eksperimen, dapat diamati bahwa ketiga algoritma dapat mendekati global optimal (state value = 0) dengan konfigurasi tertentu. Selain itu, faktor random juga berpengaruh cukup signifikan terhadap hasil akhir pencarian yang sangat bergantung pada pembangkitan random state seperti Simulated Annealing. Secara umum, ketiga jenis algoritma mampu mendekati global optimum.

Berdasarkan data yang telah diambil dari masing masing algoritma, untuk Hill Climbing, pada Steepest dan Sideways, memberikan hasil yang konsisten dan state value akhir yang maksimal, karena pada algoritma nya melakukan *looping* tak hingga sampai current value lebih baik daripada random value, namun pada kasus pengujian ini adalah unik, karena menghasilkan plot yang sama dan seluruhnya berhasil mencapai global optimum (state value = 0), tentu saja apabila steepest berhasil mencapai global optimum, maka dapat dipastikan Sideways juga akan mencapai global optimum. Sedangkan untuk Random Restart tidak terjadi restart, dan hal ini adalah mungkin apabila dilihat dari Steepest Ascent yang berhasil mencapai global optimum, sehingga sama seperti menjalankan Steepest Ascent tanpa restart. Untuk Stochastic, Iterasi yang dibutuhkan rata rata adalah ribuan, dengan generateRandomSuccessor untuk mencapai state value terbaik, seperti Steepest Ascent. Kemudian untuk Simulated Annealing, hasil pencarian dengan konfigurasi temperatur inisial 1000, kecepatan penurunan 0.95, temperatur berhenti 0.01 memberikan hasil pencarian yang baik, karena bisa mencapai hasil akhir state value = 0 (paling baik), kemudian dilanjut pada konfigurasi temperatur inisial 500, kecepatan penurunan 0.99, temperatur berhenti 0.1 memberikan hasil pencarian yang baik namun tidak mencapai state value = 0 (rata rata dengan 3 percobaan adalah -50) dan dilanjutkan dengan konfigurasi temperatur inisial 100, kecepatan penurunan 0.95, dan temperatur berhenti 1 memberikan hasil yang paling buruk diantara seluruhnya dengan state value akhir = -200, Hal ini dinilai hasil pencarian sangat bergantung kepada konfigurasi pada Simulated Annealing, apabila temperatur initial semakin tinggi dengan kecepatan penurunan yang rendah dan temperatur berhenti yang rendah maka akan menghasilkan state value akhir yang baik, karena algoritmanya akan semakin banyak iterasinya untuk mencari successor sehingga memungkinkan mendapatkan value yang lebih baik dari sebelumnya. Dengan iterasi yang besar juga dapat menemukan *stuck frequency* yang besar juga.

Untuk Genetic Algorithm, telah dilakukan beberapa pengujian, dengan jumlah populasi = 8 dengan iterasi 1000 dan 10000 dapat menghasilkan state value akhir = 0,

hal ini dikarenakan pada Genetic Algorithm, dengan iterasi yang semakin besar maka akan lebih berpeluang untuk mendapatkan hasil state value yang sangat baik, bahkan global optimum, Termasuk juga jumlah populasi, semakin besar maka lebih berpotensi untuk mendapatkan hasil yang maksimum, karena akan mengaitkan banyak parent dan dalam algoritma terdapat heuristik dimana apabila di suatu iterasi menghasilkan child yang buruk, maka akan di pruned.

Durasi ketiga jenis algoritma ini dapat digolongkan berdasarkan ketergantungannya pada jumlah iterasi. Durasi eksekusi stochastic hill climbing, simulated annealing, dan genetic algorithm dapat dikontrol melalui jumlah iterasi. Semakin banyak jumlah iterasi, semakin lambat pula durasi eksekusinya, tetapi hasil yang diperoleh semakin baik. Adapun sebagian besar algoritma hill climbing tidak dibatasi oleh waktu dan baru akan berhenti ketika mendapat hasil yang baik. Berdasarkan eksperimen, waktu yang diperlukan untuk mencapai solusi ini relatif lebih lambat dibandingkan dua jenis algoritma lainnya.

Algoritma Hill Climbing, dalam hal ini steepest ascent, sideways, dan random restart memiliki hasil yang cukup konsisten dalam mencapai hasil akhir. Adapun stochastic sangat bergantung pada jumlah iterasi. Algoritma Simulated Annealing kurang memiliki hasil yang konsisten dalam mencapai solusi, hal ini dikarenakan algoritma ini, sama seperti stochastic hill climbing, sangat bergantung pada hasil random. Adapun Genetic Algorithm mampu memberikan hasil yang cukup konsisten selaras dengan jumlah populasi dan jumlah iterasi. Pada Genetic Algorithm, banyak iterasi dan jumlah populasi per generasi memiliki dampak yang cukup signifikan terhadap hasil akhir pencarian. Jumlah iterasi yang terlalu sedikit membuat populasi kurang mendapat kesempatan untuk memperbaiki generasinya. Adapun jumlah populasi per generasi berdampak positif dalam memperbanyak gen pool yang baik sehingga dapat menghasilkan solusi dengan lebih cepat.

BAB IV

KESIMPULAN DAN SARAN

4.1 Kesimpulan

Permasalahan penjadwalan mata kuliah dapat diselesaikan dengan algoritma local search, seperti algoritma Hill Climbing, Simulated Annealing, dan Genetic Algorithm. Ketiga algoritma ini memiliki karakteristik yang beragam. Algoritma stochastic hill climbing dan simulated annealing memiliki waktu eksekusi yang relatif cepat, tetapi sangat bergantung pada hasil random dan jumlah iterasi. Sebagian besar algoritma hill climbing seperti steepest ascent dan sideways memiliki waktu eksekusi yang relatif lambat, tetapi mampu menjamin global optima dengan cukup baik pada kasus ini. Adapun genetic algorithm yang bergantung hasil random dan jumlah iterasi, tetap dapat memberikan hasil akhir yang cukup baik karena berusaha menjamin hasil random yang terus membaik.

4.2 Saran

- Apabila berkeinginan untuk mendapatkan hasil yang lebih baik, maka konfigurasi jumlah iterasinya lebih banyak, terutama pada Simulated Annealing (Temperature inisial dan Cooling Rate) dan Genetic Algorithm, ini akan memberikan peluang lebih untuk mendapatkan hasil state value yang paling baik, namun dengan *trade off* melalui waktu eksekusi yang lama. Namun apabila mengutamakan pada waktu eksekusi maka, konfigurasi jumlah iterasi lebih sedikit, dengan *trade off*, hasil akhir mungkin kurang maksimal.
- Apabila perlu, tambahkan constraint dan heuristik sehingga algoritma dapat berjalan dengan cepat serta menghasilkan hasil akhir yang sesuai, seperti yang telah diimplementasikan disini adalah pruning.

LAMPIRAN

Link github : https://github.com/rlukassa/AI_anak_indonesia

Tabel Pemenuhan Spesifikasi

No	Spesifikasi	Terpenuhi	Tidak Terpenuhi
1	Mengerjakan tepat waktu dengan sabar, sepuh hati dan menjunjung tinggi kejujuran.	😎	
2	Mengimplementasi Algoritma Hill Climbing Steepest Ascent	😎	
3	Mengimplementasi Algoritma Simulated Annealing	😎	
4	Mengimplementasi Algoritma Genetic Algorithm	😎	
5	Mengimplementasi 4 objective function berbeda	😎	
6	Program dapat berjalan tanpa error dan bug	😎	
7	Program dapat menampilkan state awal, dan state akhir yang menjadi solusi.	😎	
8	Program dapat menghasilkan solusi yang optimal sesuai algoritmanya.	😎	
9	Melakukan seluruh pengujian sesuai yang tertera pada dokumen spek.	😎	
10	Menulis laporan dalam bahasa Indonesia yang baik dan benar sesuai kaidah PUEBI dan kosa kata KBBI.	😎	
11	BONUS: Mengimplementasi seluruh algoritma hill climbing (sideways, stochastic, random restart).	😎	
12	BONUS: Menambah Dosen sebagai perhitungan dalam mempertimbangkan penjadwalan.	😎	

Tabel Pembagian Kerja

NIM	Nama	Pembagian Kerja
13523126	Brian Ricardo Tamin	- Membuat kelas entities dan JSON parser - Membuat seluruh algoritma Hill Climbing

		- Mengerjakan laporan
13523144	Muhamad Nazih Najmudin	- Membuat kelas State dan objective function - Membuat Genetic Algorithm - Mengerjakan laporan
13523158	Lukas Raja Agripa	- Membuat visualisasi state dan main program - Membuat algoritma Simulated Annealing - Mengerjakan laporan

REFERENSI

- [1] Stanford University & Berkeley AI Materials, “AIMA — The Berkeley Artificial Intelligence Materials,” Available: <https://aima.cs.berkeley.edu/> , accessed: Oct. 17, 2025.
- [2] Module Artificial Intelligence ITB Edunex — Beyond Classical Search Available: <https://edunex.itb.ac.id/> , accessed: Oct. 17, 2025